

The PB-ADV bearer allows provisioning of an unprovisioned device by using an advertising bearer. The PB-GATT bearer allows provisioning of an unprovisioned device by using the Mesh Provisioning Service.

The PB-Remote bearer allows a Provisioner that is outside immediate radio range of an unprovisioned device to communicate with a node supporting the Remote Provisioning Server model that is within immediate radio range of the unprovisioned device and to use that node as a retransmitter to communicate with the unprovisioned device using PB-ADV or PB-GATT. This allows a Provisioner to provision unprovisioned devices using nodes of the mesh network. [Figure 5.2](#) illustrates this process when the Remote Provisioning Server uses PB-ADV to provision an unprovisioned device.

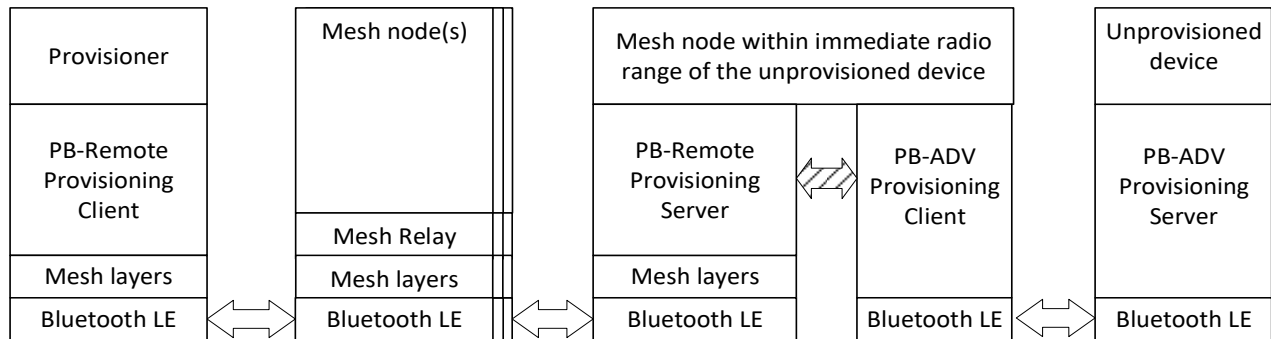


Figure 5.2: Devices participating in provisioning using PB-Remote and PB-ADV

The Provisioner may use the Remote Provisioning Server (see [Section 4.4.5](#)) to identify unprovisioned devices within immediate radio range of the server.

As a final step, the Provisioner closes the provisioning bearer. Some provisioning bearers require that a close reason be provided when closing the bearer. The close reason is provided by the provisioning protocol. When the provisioning bearer is closed by the peer device, the close reason is provided to the provisioning protocol.

5.1 Endianness

Unless stated otherwise, all multiple-octet numeric values in this layer shall be big-endian, as described in [Section 3.1.1.1](#).

5.2 Provisioning bearer layer

A provisioning bearer layer enables the transport of Provisioning PDUs between a Provisioner and an unprovisioned device. Three provisioning bearers are defined:

- PB-ADV (see [Section 5.2.1](#))
- PB-GATT (see [Section 5.2.2](#))
- PB-Remote (see [Section 5.2.3](#))

Six provisioning roles are defined:

- PB-ADV Client
- PB-ADV Server
- PB-GATT Client

- PB-GATT Server
- PB-Remote Client
- PB-Remote Server

An unprovisioned device may support the PB-ADV Server role and may support the PB-GATT Server role. An unprovisioned device should support both the PB-ADV Server role and the PB-GATT Server role.

A Provisioner shall support either the PB-ADV Client role or PB-GATT Client role, or both roles. If the Provisioner supports only one role, the Provisioner should support the PB-ADV Client role. The Provisioner may also support the PB-Remote Client role.

A node may support the PB-Remote Server role.

5.2.1 PB-ADV

PB-ADV is a provisioning bearer used to provision an unprovisioned device using Generic Provisioning PDUs (see Section 5.3) over the advertising channels. The provisioning mechanism is session based. An unprovisioned device shall support only one session at a time. There is no such limitation for a Provisioner. A session is established using the Link Establishment procedure (see Section 5.3.2). The PB-ADV provisioning bearer requires a reason when closing the link and provides a reason when the link is closed.

When PB-ADV is used, the Provisioner shall use the PB-ADV Client role and the unprovisioned device shall use the PB-ADV Server role.

The PB-ADV bearer is used for transmitting Generic Provisioning PDUs. The PB-ADV bearer maximum transmission unit (MTU) size is 24 octets.

When using PB-ADV, a Generic Provisioning PDU shall be sent using the PB-ADV AD type identified by «PB-ADV», as defined in [4].

A Provisioner and an unprovisioned device supporting PB-ADV should perform passive scanning with a duty cycle as close to 100 percent as possible in order to avoid missing any incoming PB-ADV PDUs.

The PB-ADV AD type contains a PB-ADV PDU field. The format of the PB-ADV AD type is defined in Table 5.1.

Field	Size (octets)	Description	Req.
Length	1	Length of the AD Type and PB-ADV PDU fields	M
AD Type	1	«PB-ADV»	M
PB-ADV PDU	variable	PB-ADV PDU	M

Table 5.1: PB-ADV AD type

Any advertising data that includes the PB-ADV AD type shall use ADV_NONCONN_IND PDUs. If a node receives a PB-ADV AD type in a PDU type different from ADV_NONCONN_IND, the message shall be ignored.

The format of the PB-ADV PDU field is defined in [Table 5.2](#).

Field	Size (octets)	Description	Req.
Link ID	4	The identifier of a link	M
Transaction Number	1	The number for identifying a transaction	M
Generic Provisioning PDU	1–24	Generic Provisioning PDU being transferred	M

Table 5.2: PB-ADV PDU field format

The Link ID is used to identify a link between two devices.

The Transaction Number field contains a one-octet value used to identify each individual Generic Provisioning PDU sent by the device. When a Provisioning PDU that does not fit in a single PB-ADV PDU is segmented, all segments are sent using the same Transaction Number field value. When a Provisioning PDU is retransmitted, the Transaction Number field is not changed.

Transport-specific messages are defined to establish and terminate the link between two devices (see [Section 5.3.1.4](#)). When the link is opened, the Provisioner and the unprovisioned device shall start the link timer from the initial value set to 60 seconds. When the link timer expires, the Provisioner and the unprovisioned device shall close the link. When the unprovisioned device receives any Provisioning PDU or a Link Close message, then the unprovisioned device shall stop the link timer.

The Generic Provisioning PDU field format is defined in [Section 5.3](#).

The following rules shall be implemented when sending a PB-ADV PDU:

- When the Generic Provisioning PDU field contains a Provisioning Bearer Control PDU, the Transaction Number field shall be set to 0 and ignored upon reception.
- When the PB-ADV Client is sending a Provisioning PDU for the first time over an open provisioning link, it shall start with a Transaction Number field value of 0x00. The PB-ADV Client shall increment the field value by 1 for each new Provisioning PDU it is sending for the duration of the provisioning link. If the field value has reached 0x7F, it shall wrap to 0x00 on sending the next Provisioning PDU.
- When the PB-ADV Server is sending a Provisioning PDU for the first time over an open provisioning link, it shall start with a Transaction Number field value of 0x80. The PB-ADV Server shall increment the field value by 1 for each new Provisioning PDU it is sending for the duration of the provisioning link. If the field value has reached 0xFF, it shall wrap to 0x80 on sending the next Provisioning PDU.
- When a PB-ADV Server is receiving a Provisioning PDU, it shall set the Transaction Number field to the value of the Transaction Number field of the PB-ADV PDUs being received during the transaction.
- When a PB-ADV Server is sending a Transaction Acknowledgement PDU, the Transaction Number field shall be set to the value of the Transaction Number field of the PB-ADV PDUs transporting the Provisioning PDU being acknowledged.

5.2.2 PB-GATT

PB-GATT is a provisioning bearer used to provision an unprovisioned device using Proxy PDUs (see Section 6.3) to encapsulate Provisioning PDUs (see Section 5.4) within the Mesh Provisioning Service (see Section 7.1). PB-GATT is provided for support when a Provisioner does not support PB-ADV due to limitations of the application interfaces.

The PB-GATT Server shall use the Provisioning Server role (see Section 6.2.2) and the PB-GATT Client shall use the Provisioning Client role (see Section 6.2.2).

When PB-GATT is used, the Provisioner shall use the PB-GATT Client role and the unprovisioned device shall use the PB-GATT Server role.

To enable very low power operation for the PB-GATT Server and allow the PB-GATT Server to calculate the Diffie-Hellman shared secret without requiring the energy to maintain an idle link, the connection interval for the connection between a PB-GATT Client and a PB-GATT Server should be between 250 and 1,000 milliseconds.

The PB-GATT Server shall be able to receive a single Proxy PDU (see Section 6.3) in a single Write Command ATT PDU.

The PB-GATT Server shall use a single Handle Value Notification ATT PDU to send Provisioning PDUs to a PB-GATT Client.

If the negotiated ATT_MTU is smaller than a required Proxy PDU size, the transmission of the Mesh Provisioning Data In and Mesh Provisioning Out characteristics always needs to be fragmented and reassembled. Each PDU shall be fully reassembled before processing.

The PB-GATT Server shall be able to receive a Proxy PDU in one or several ATT PDUs. The PB-GATT Server shall use one or several Handle Value Notification ATT PDUs to send a Proxy PDU to the PB-GATT Client depending on the size of the message and the negotiated ATT_MTU.

The Mesh Provisioning Data In and Mesh Provisioning Data Out characteristic formats use the Proxy PDU format defined in Section 6.3.1.

The PB-GATT provisioning bearer does not require a reason when closing the link and does not provide a reason when the link is closed.

Figure 5.3 illustrates the provisioning protocol, PB-GATT Server, and Mesh Provisioning Service (see Section 7.1) interactions.

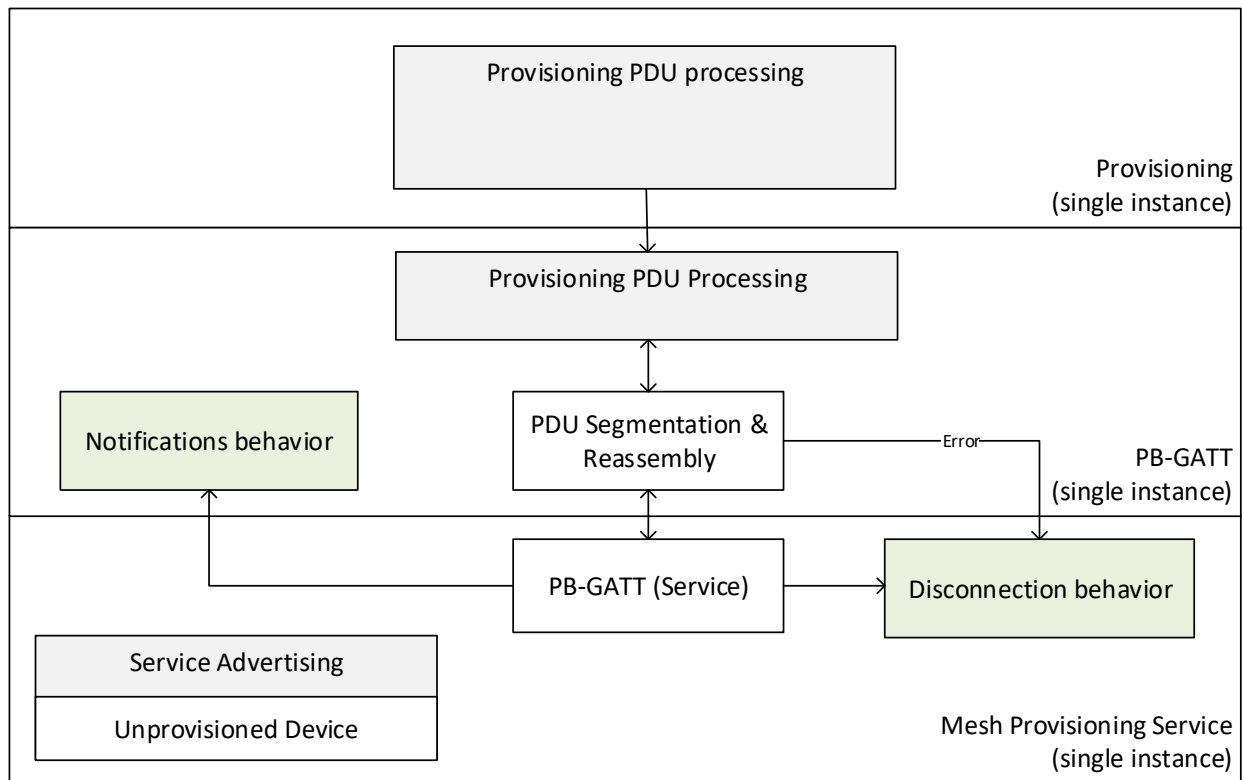


Figure 5.3: Provisioning protocol, PB-GATT Server, and Mesh Provisioning Service interactions

The link is opened on a PB-GATT bearer when the PB-GATT Client enables notifications (by writing 0x0001) to the Mesh Provisioning Data Out Client Characteristic Configuration Descriptor after a connection is established (see Section 7.1.3.2.1). When the link is opened, the Provisioner and the unprovisioned device shall start the link timer from the initial value set to 60 seconds. When the link timer expires, the Provisioner and the unprovisioned device shall disconnect. When the Provisioner receives any Provisioning PDU or the connection between the PB-GATT Client and the PB-GATT Server closes, then the Provisioner shall stop the link timer. When the unprovisioned device receives any Provisioning PDU or the connection between the PB-GATT Client and the PB-GATT Server closes, then the unprovisioned device shall stop the link timer.

5.2.3 PB-Remote

The PB-Remote provisioning bearer uses the existing mesh network to provision an unprovisioned device that is not within immediate radio range of the Provisioner. PB-Remote uses the PB-ADV bearer (see Section 5.2.1) or the PB-GATT bearer (see Section 5.2.2) for the last hop to the unprovisioned device. PB-Remote uses one of the mesh nodes as a PB-Remote Server to manage the PB-ADV or PB-GATT bearer link on behalf of the Provisioner.

Provisioning should be done using an OOB public key when PB-Remote is used. Provisioning should not be done using Authentication with No OOB when PB-Remote is used.

When PB-Remote is supported by the Provisioner, the Provisioner shall use the PB-Remote Client role. When the node supports the Remote Provisioning Server model, the node uses the PB-Remote Server role.

PB-Remote may also be used to execute the Device Key Refresh procedure (see Section 3.11.8.4) or the Node Address Refresh procedure (see Section 3.11.8.5) or the Node Composition Refresh procedure (see Section 3.11.8.6) between the Provisioner (PB-Remote Client) and the PB-Remote Server.

Multiple instances of the PB-Remote Client can be used by the Provisioner to communicate with many nodes implementing the PB-Remote Server, thus providing the capability to provision many unprovisioned devices at the same time. The PB-Remote Server can only communicate with one PB-Remote Client and can only open one supported provisioning bearer at a time.

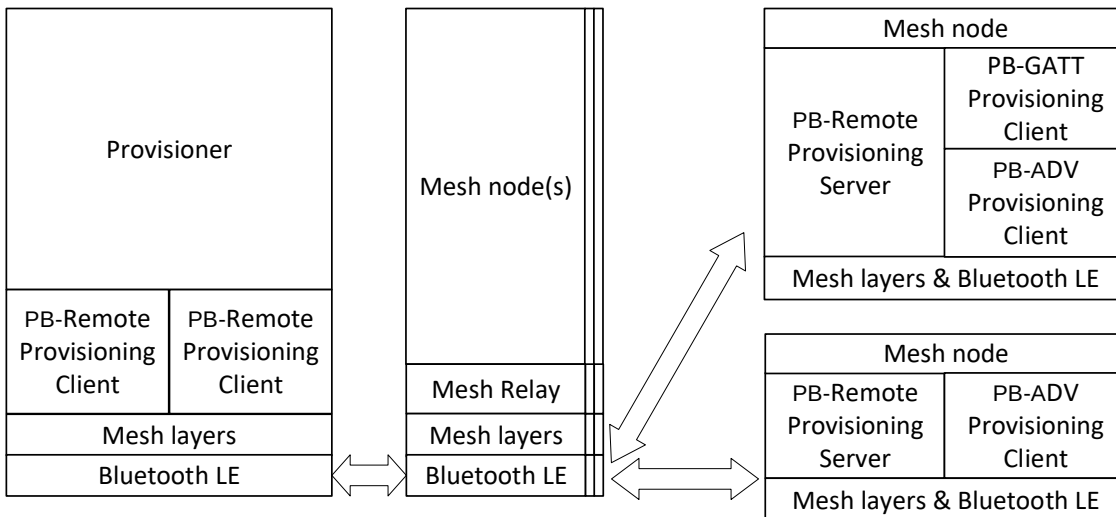


Figure 5.4: A Provisioner using multiple instances of the PB-Remote Client

The PB-Remote provisioning bearer requires a reason when closing the link. When a PB-Remote Server uses the PB-ADV bearer for the last hop to the device being provisioned and the PB-ADV link is closed, the PB-Remote bearer provides a close reason. When the PB-Remote Server uses the PB-GATT bearer, the provided reason is not used when closing the link. The PB-Remote Client is not aware whether the PB-ADV or PB-GATT bearer is used by the PB-Remote Server, so providing a reason is mandatory.

5.2.3.1 PB-Remote Client

The PB-Remote Client is a provisioning device that controls the provisioning process. The PB-Remote Client uses the Remote Provisioning Client model. The PB-Remote Client can choose which PB-Remote Server device to communicate with, can instruct the PB-Remote Server to start scanning for unprovisioned devices, and can instruct the PB-Remote Server to establish a provisioning link with the chosen unprovisioned device. After the Provisioning Bearer link is established, the PB-Remote Client runs the provisioning protocol by executing the PB-Remote's Send PDU procedure (see Section 5.2.3.3.3) and the PB-Remote Server executes the PB-Remote's Receive PDU procedure (see Section 5.2.3.3.4).

5.2.3.2 PB-Remote Server

The PB-Remote Server is a mesh node that supports the Remote Provisioning Server model. The PB-Remote Server uses the Remote Provisioning Scan procedure to scan for unprovisioned devices. The server uses the Remote Provisioning Extended Scan procedure to obtain additional information about unprovisioned devices and uses the Provisioning Bearer protocol to establish a Provisioning Bearer link

with an unprovisioned device. After a Provisioning Bearer link is established, the PB-Remote Server transports Provisioning PDUs between the PB-Remote Client and the connected Provisioning Bearer protocol.

5.2.3.3 PB-Remote procedures

The PB-Remote Server may support multiple provisioning bearers, but the server shall use only one provisioning bearer at a time. Each provisioning bearer defines different steps that are needed to open or close the bearer connection and to send or receive the Provisioning PDU over the bearer. The subsections 5.2.3.3.1 through 5.2.3.3.4 define common names for these bearer specific steps, thus defining the PB-Remote procedures.

5.2.3.3.1 PB-Remote Open Link procedure

The PB-Remote Open Link procedure is used to establish a connection between the PB-Remote Server and the unprovisioned device. The PB-Remote Server initializes the connection. The procedure accepts the UUID of the device that the provisioning bearer will be open to as a Device UUID parameter and an optional Timeout parameter. The Timeout parameter is expressed in seconds and the valid range is from 1 to 60 seconds with one second granularity. The default value of the Timeout parameter is 10 seconds. The procedure succeeds or fails depending on the outcome of the opening of a provisioning bearer. The time before the procedure fails via timeout shall be set to the value of the Timeout parameter.

Table 5.3 defines Success and Fail results for the PB-Remote Open Link procedure.

Provisioning Bearer	Procedure Success	Procedure Fail
PB-ADV	The Link Establishment procedure (see Section 5.3.2) establishes a session.	The Link Establishment procedure does not establish a session.
PB-GATT	The PB-GATT Client successfully opens a connection to the PB-GATT Server (see Section 5.2.2).	The PB-GATT Client is unable to open a connection to the PB-GATT Server.

Table 5.3: PB-Remote Open Link procedure results

5.2.3.3.2 PB-Remote Close Link procedure

The PB-Remote Close Link procedure is used to close a connection between a PB-Remote Server and the unprovisioned device. The procedure accepts one parameter: Reason. Some provisioning bearers require the Reason parameter to close the connection. This procedure always succeeds. Table 5.4 defines results for the PB-Remote Close Link procedure for each provisioning bearer.

Provisioning bearer	Procedure
PB-ADV	Link close step of the Link Establishment procedure (see Section 5.3.2).
PB-GATT	Closing a connection between a PB-GATT Client and a PB-GATT Server (see Section 5.2.2).

Table 5.4: PB-Remote Close Link procedure results

5.2.3.3.3 PB-Remote Send PDU procedure

The PB-Remote Send PDU procedure is used to send a Provisioning PDU from the PB-Remote Server over an open provisioning bearer to the unprovisioned device. The procedure accepts one parameter: Provisioning PDU. The procedure can either succeed or fail depending on the outcome of the Provisioning PDU delivery.

Table 5.5 defines the results for the PB-Remote Send PDU procedure.

Provisioning Bearer	Procedure Success	Procedure Fail
PB-ADV	The Provisioning PDU is delivered successfully from the Provisioning Server to the unprovisioned device using the PB-ADV provisioning bearer (see Section 5.3.3)	The Provisioning PDU delivery from the Provisioning Server to the unprovisioned device using the PB-ADV bearer fails.
PB-GATT	The Provisioning PDU is delivered successfully from the PB-Remote Server acting as the PB-GATT Client to the unprovisioned device (PB-GATT Server) using the PB-GATT bearer (see Section 5.2.2).	The Provisioning PDU delivery from the PB-Remote Server to the unprovisioned device using the PB-GATT bearer fails.

Table 5.5: PB-Remote Send PDU procedure

5.2.3.3.4 PB-Remote Receive PDU procedure

The PB-Remote Receive PDU procedure is used to receive a Provisioning PDU sent from the unprovisioned device to the PB-Remote Server over an open provisioning bearer. When the PB-ADV bearer is used, the PB-Remote Server uses the PB-ADV Client for this procedure. When the PB-GATT bearer is used, the server uses the PB-GATT Client. The procedure either succeeds or fails depending on the outcome of the PDU reception by the PB-Remote Server. The output of the procedure is the received Provisioning PDU.

The procedure is canceled when the open provisioning bearer is closed.

Table 5.6 defines the results for the PB-Remote Receive PDU procedure.

Provisioning Bearer	Procedure Success	Procedure Fail
PB-ADV	The Provisioning PDU is delivered successfully from the unprovisioned device to the PB-Remote Server using the PB-ADV bearer (see Section 5.3.3).	The Provisioning PDU delivery from the unprovisioned device to the PB-Remote Server using the PB-ADV bearer fails.
PB-GATT	The Provisioning PDU is delivered successfully to the PB-Remote Server acting as the PB-GATT Client from the unprovisioned device (PB-GATT Server) using the PB-GATT bearer (see Section 5.2.2).	The Provisioning PDU delivery from unprovisioned device to the PB-Remote Server using the PB-GATT bearer fails.

Table 5.6: PB-Remote Receive PDU procedure

5.3 Generic provisioning layer

The generic provisioning layer is responsible for transport of Generic Provisioning PDUs over an unreliable connectionless provisioning bearer. This layer also defines Generic Provisioning PDUs.

The Generic Provisioning PDU format consists of a Generic Provisioning Control (GPC) field followed by a variable-length Generic Provisioning Payload field as illustrated in Figure 5.5 and defined in Table 5.7.

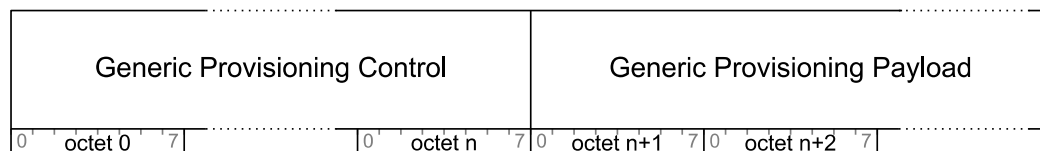


Figure 5.5: Generic Provisioning PDU format

Field	Size (octets)	Description	Req.
Generic Provisioning Control	1–17	Generic Provisioning Control field	M
Generic Provisioning Payload	0–64	Generic Provisioning Payload (segments of the Provisioning PDU)	M

Table 5.7: Generic Provisioning PDU format

The two least significant bits of the first octet of the Generic Provisioning Control field contain a Generic Provisioning Control Format (GPCF) field that determines the format of the Generic Provisioning Control field. The GPCF field is an enumeration with the values shown in Table 5.8.

Value	Description
0b00	Transaction Start
0b01	Transaction Acknowledgment
0b10	Transaction Continuation

Value	Description
0b11	Provisioning Bearer Control

Table 5.8: Generic Provisioning Control Format field values

The format of the GPC field for each format type is defined in Section 5.3.1.

5.3.1 Generic Provisioning PDU types

5.3.1.1 Transaction Start PDU

A Transaction Start PDU is used to start the transmission of a segmented message. The Generic Provisioning Control field of a Transaction Start PDU is illustrated in Figure 5.6 and shown in Table 5.9.

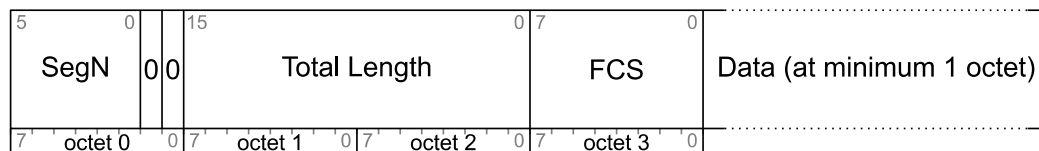


Figure 5.6: Transaction Start PDU

Field	Size (bits)	Description	Req.
SegN	6	The last segment number	M
GPCF	2	Set to Transaction Start	M
TotalLength	16	The number of octets in the Provisioning PDU	M
FCS	8	Frame Check Sequence of the Provisioning PDU	M

Table 5.9: Generic Provisioning Control field for Transaction Start PDU

The SegN field shall be set to the last segment number (zero-based) of this transaction.

The GPCF field shall be set to Transaction Start.

The TotalLength field shall be set to the number of octets in the Provisioning PDU.

When transmitted using PB-ADV, the FCS field is calculated as defined by 3GPP TS 27.010 with the Polynomial ($x^8 + x^2 + x^1 + 1$) and is calculated over the Provisioning PDU only.

The Generic Provisioning Payload shall contain segment 0 of the Provisioning PDU.

5.3.1.2 Transaction Acknowledgment PDU

A Transaction Acknowledgment PDU is used to acknowledge a Provisioning PDU. The Generic Provisioning Control field of a Transaction Acknowledgment PDU is illustrated in Figure 5.7 and shown in Table 5.10.

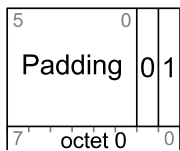


Figure 5.7: Transaction Acknowledgment PDU

Field	Size (bits)	Description	Req.
Padding	6	0b000000; all other values Prohibited	M
GPCF	2	Set to Transaction Acknowledgment	M

Table 5.10: Generic Provisioning Control field for Transaction Acknowledgment PDU

The GPCF field shall be set to Transaction Acknowledgment.

The Generic Provisioning Payload is zero length.

5.3.1.3 Transaction Continuation PDU

A Transaction Continuation PDU is used to send additional segments of a Provisioning PDU. The Generic Provisioning Control field of a Transaction Continuation PDU is illustrated in Figure 5.8 and shown in Table 5.11.

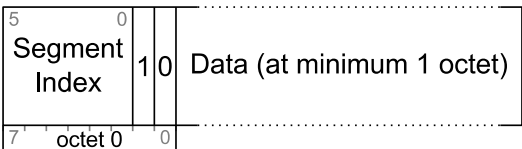


Figure 5.8: Transaction Continuation PDU

Field	Size (bits)	Description	Req.
SegmentIndex	6	Segment number of the transaction	M
GPCF	2	Set to Transaction Continuation	M

Table 5.11: Generic Provisioning Control field for Transaction Continuation PDU

The SegmentIndex field shall be set to the segment number contained within this PDU.

The GPCF field shall be set to Transaction Continuation.

The Generic Provisioning Payload shall contain segment SegmentIndex of the Provisioning PDU.

5.3.1.4 Provisioning Bearer Control

A Provisioning Bearer Control PDU is used to manage sessions on bearers that have no inherent session management. The Generic Provisioning Control field of a Provisioning Bearer Control PDU is illustrated in Figure 5.9 and shown in Table 5.12. The Provisioning Bearer Control PDUs are defined in the following sections.

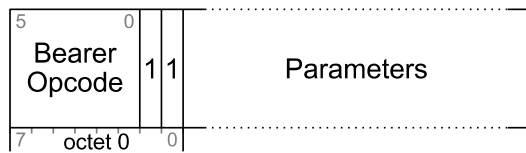


Figure 5.9: Provisioning Bearer Control PDU

Field	Size (bits)	Description	Req.
BearerOpcode	6	The opcode for the provisioning bearer control PDUs	M
GPCF	2	Set to Provisioning Bearer Control	M
Parameters	variable	Parameters defined by each BearerOpcode	M

Table 5.12: Generic Provisioning Control field for Provisioning Bearer Control PDU

The BearerOpcode values are defined in [Table 5.13](#).

Value	Message	Description
0x00	Link Open	Open a session on a bearer with an unprovisioned device
0x01	Link ACK	Acknowledge a session on a bearer
0x02	Link Close	Close a session on a bearer
0x03–0x3F	RFU	Reserved for Future Use

Table 5.13: BearerOpcode field values

The GPCF field shall be set to Provisioning Bearer Control.

The Generic Provisioning Payload is zero length.

The Parameters of each message are defined in the sections that follow.

5.3.1.4.1 Link Open message

A Link Open message is used to establish a link between a Provisioner and an unprovisioned device. The unprovisioned device shall acknowledge this message with the Link ACK message.

The Parameters field for the Link Open message is defined in [Table 5.14](#) and the message is illustrated in [Figure 5.10](#).

Field	Size (octets)	Description	Req.
Device UUID	16	This is the Device UUID of the chosen unprovisioned device	M

Table 5.14: Parameters field of Link Open message

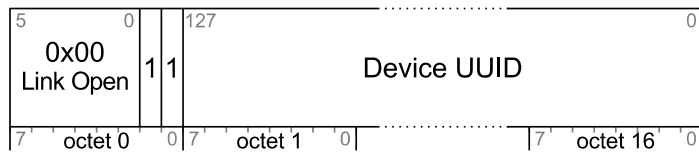


Figure 5.10: Link Open message format

5.3.1.4.2 Link ACK message

A Link ACK message is sent to acknowledge the receipt of the Link Open message.

There are no Parameters for this message.

The Link ACK message is illustrated in Figure 5.11.

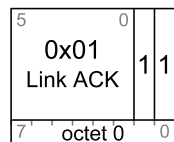


Figure 5.11: Link ACK message format

5.3.1.4.3 Link Close message

A Link Close message is used to close a link. Since this message is not acknowledged, the sender shall repeat this message at least three times. Both sides of the link may send this message. This message shall be accepted and processed regardless of the setting of the Reason field.

The Parameters field for the Link Close message is defined in Table 5.15 and the message is illustrated in Figure 5.12.

Field	Size (octets)	Description	Req.
Reason	1	The reason for closing the link	M

Table 5.15: Parameters field of Link Close message

The Reason field values are defined in Table 5.16.

Value	Reason	Description
0x00	Success	The provisioning was successful
0x01	Timeout	The provisioning transaction timed out
0x02	Fail	The provisioning failed
0x03–0xFF	Unrecognized	Unrecognized reason that may be defined in the future.

Table 5.16: Reason field values

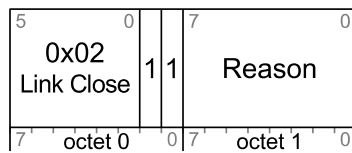


Figure 5.12: Link Close message format

5.3.2 Link Establishment procedure

The Link Establishment procedure is used to establish a session for a bearer that does not have inherent session management. A session is identified by using a Link ID that is static for the duration of the session and shall be randomly generated to prevent collisions between sessions.

A link is established between a Provisioner and an unprovisioned device for sending provisioning messages. The unprovisioned device is identified by a Device UUID (see Section 3.11.3).

The Provisioner shall scan for unprovisioned devices. Upon receiving an Unprovisioned Device beacon, the Provisioner may establish a link with the unprovisioned device identified by the Device UUID.

The link is open for the unprovisioned device when the unprovisioned device sends a Link ACK for the Link Open message. The link is open for the Provisioner when a Link ACK is received with the Link ID equal to the Link ID sent in the Link Open message. The unprovisioned device is being provisioned when the link is open and the unprovisioned device has received any Provisioning PDU.

When the link is open, the unprovisioned device shall ignore Generic Provisioning PDUs with the Link ID not equal to the Link ID received in the Link Open message.

When the link is not open, and the unprovisioned device receives a Link Open message, then the unprovisioned device shall accept this message by replying with a Link ACK message with the same Link ID set in the PB-ADV PDU. When the link is open, and the unprovisioned device is not being provisioned, and the unprovisioned device receives a Link Open message with the same Link ID, then the unprovisioned device shall reply with a Link ACK message with the same Link ID set in the PB-ADV PDU. When the link is open, the unprovisioned device shall ignore all Provisioning Bearer Control PDUs received with a different Link ID.

To open a link, the Provisioner shall start the link establishment timer from the initial value set to 60 seconds, and then shall start sending Link Open messages. The Provisioner may cancel this procedure at any time. The Link Open message contains the Device UUID of the unprovisioned device. On PB-ADV, the PB-ADV PDU format includes a Link ID field.

When the link establishment timer expires, the link is considered not open and the procedure may be restarted. When the link is open, the Provisioner shall stop the link establishment timer.

The link may be closed at any time after link establishment by sending the Link Close message. Either side of the link may send the Link Close message.

The Link Open, Link ACK, and Link Close messages are defined in Section 5.3.1.4.

The message sequence for establishment of a link by ID between a Provisioner and an unprovisioned device is illustrated by Figure 5.13 below.

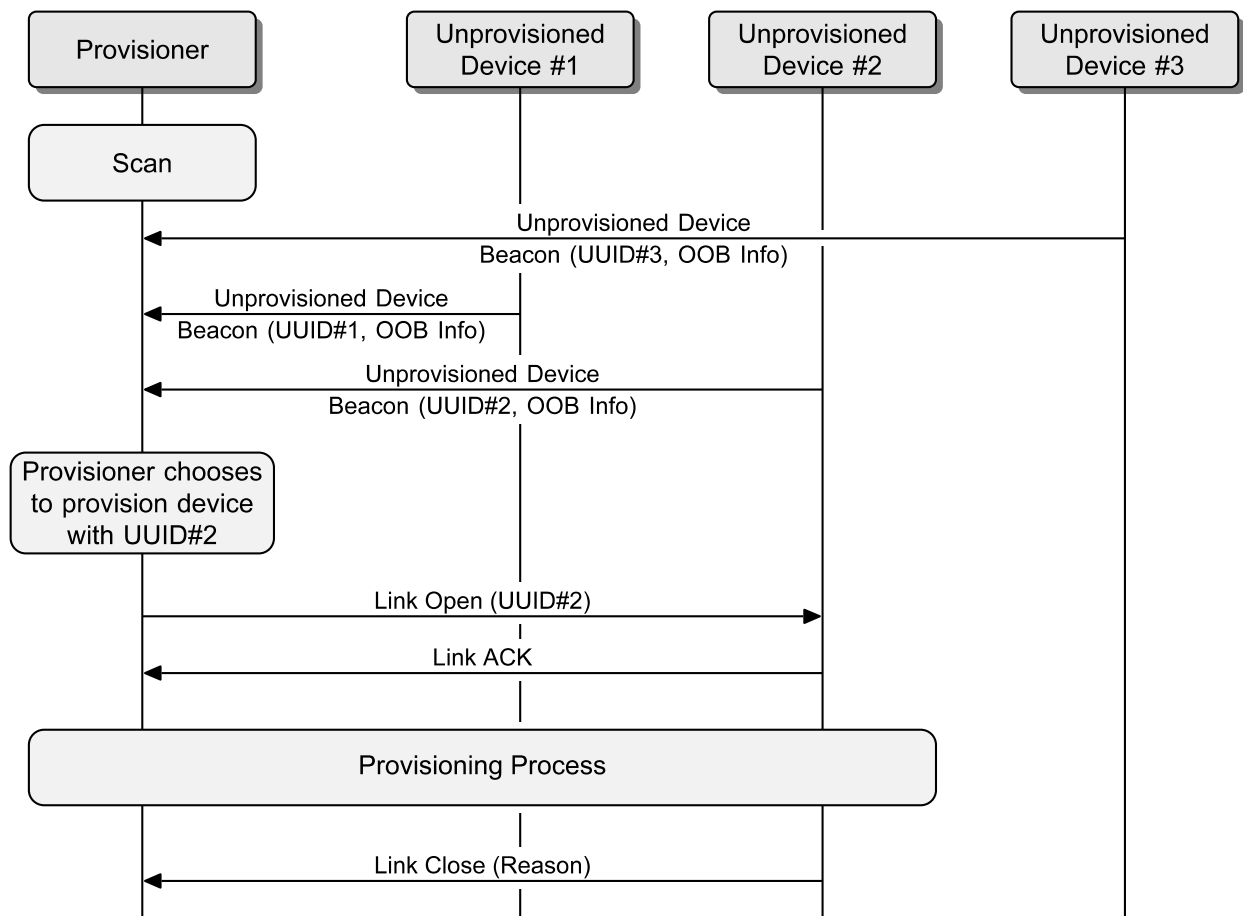


Figure 5.13: Establishment of Link by ID between a Provisioner and an unprovisioned device

5.3.3 Generic Provisioning behavior

Each Generic Provisioning PDU shall be sent after a random delay between 20 and 50 milliseconds.

Each Provisioning PDU (see Section 5.4) is transmitted as a separate transaction. A transaction may be composed of one or more segments.

The number of segments required to send a Provisioning PDU is determined by the size of the Provisioning PDU. Segments are indexed from 0 to 63. Segment 0 shall be sent using a Transaction Start PDU. All other segments shall be sent using a Transaction Continuation PDU. Each segment of the Provisioning PDU is placed into the Generic Provisioning Payload field of the respective Generic Provisioning PDU.

Each bearer has its own constraints on the maximum size of a Generic Provisioning PDU that can be transmitted by that bearer. Each Generic Provisioning PDU shall be the length of the full MTU for that bearer, except for the last segment of a transaction.

The sender shall send all segments of a transaction in sequence. If the sender does not receive a Transaction Acknowledgment message, the sender shall retransmit all segments of a transaction.

If the sender receives a Transaction Acknowledgment message, then the transaction has completed.

If the sender receives a message with other PDU types, then the message shall be ignored.

When the first message in a transaction is sent, the sender shall start the acknowledgment timer with an initial value of 30 seconds. When the Transaction Acknowledgment message for the transaction is received, the sender shall stop the acknowledgment timer. When the acknowledgment timer expires, the sender shall cancel the transaction, cancel the provisioning process, and close the link.

The receiver shall determine the number of segments for a transaction from the Transaction Start PDU.

On the PB-ADV bearer, when the receiver has received all segments of a transaction, the receiver shall calculate the FCS for the received Provisioning PDU, and if it matches the FCS field in the Transaction Start PDU, it shall send a Transaction Acknowledgment PDU after a random delay between 20 and 50 milliseconds.

When a Transaction Acknowledgment PDU has been sent for a given transaction and another segment of the same transaction has been received, another Transaction Acknowledgment PDU shall be sent and the received segment shall be ignored.

5.4 Provisioning protocol

This section defines requirements for Provisioning PDUs, behavior, and security.

The provisioning protocol defines two roles:

- Provisioner
- Provisionee

The Provisioner is a device that initiates the provisioning protocol. The Provisionee is a device that participates in the execution of the provisioning protocol with the Provisioner, upon request from the Provisioner.

5.4.1 Provisioning PDUs

The Provisioning PDUs are used to communicate between a Provisioner and a Provisionee.

The first octet of the Provisioning PDU is the Type field and defines the format of the Parameters of the Provisioning PDU.

The Provisioning PDU format is defined in [Table 5.17](#) and illustrated in [Figure 5.14](#).

Field	Size (bits)	Description	Req.
Padding	2	0b00. All other values are Prohibited	M
Type	6	Provisioning PDU Type value (see the Assigned Numbers document [4] Error! Reference source not found.)	M

Field	Size (bits)	Description	Req.
Parameters	variable	Message parameters	M

Table 5.17: Provisioning PDU format

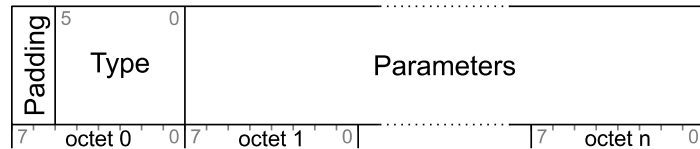


Figure 5.14: Provisioning PDU format

The Provisioning PDU Type values are defined in the Assigned Numbers document [4].

5.4.1.1 Provisioning Invite

A Provisioner sends a Provisioning Invite PDU to indicate to the intended Provisionee that the provisioning process is starting. The format of the parameters for the Provisioning Invite PDU is defined in [Table 5.18](#).

Field	Size (octets)	Description	Req.
Attention Duration	1	Attention Timer state (See Section 4.2.10)	M

Table 5.18: Provisioning Invite PDU parameters format

5.4.1.2 Provisioning Capabilities

The Provisionee sends a Provisioning Capabilities PDU to indicate its supported provisioning capabilities to a Provisioner. The format of the parameters for the Provisioning Capabilities PDU is defined in [Table 5.19](#).

Field	Size (octets)	Description	Req.
Number of Elements	1	Number of elements supported by the Provisionee	M
Algorithms	2	Supported algorithms and other capabilities (see Table 5.21)	M
Public Key Type	1	Supported public key types (see Table 5.22)	M
OOB Type	1	Supported OOB Types (see Table 5.23)	M
Output OOB Size	1	Maximum size of Output OOB supported (see Table 5.24)	M
Output OOB Action	2	Supported Output OOB Actions (see Table 5.25)	M
Input OOB Size	1	Maximum size in octets of Input OOB supported (see Table 5.26)	M
Input OOB Action	2	Supported Input OOB Actions (see Table 5.27)	M

Table 5.19: Provisioning Capabilities PDU parameters format

The Number of Elements values are defined in [Table 5.20](#).

Value	Description
0x00	Prohibited
0x01–0xFF	Number of elements supported by the Provisionee

Table 5.20: Number of Elements field values

The Algorithms values are defined in [Table 5.21](#).

Bit	Name
0	BTM_ECDH_P256_CMAC_AES128_AES_CCM
1	BTM_ECDH_P256_HMAC_SHA256_AES_CCM
2–15	Reserved for Future Use

Table 5.21: Algorithms field values

The BTM_ECDH_P256_HMAC_SHA256_AES_CCM bit of the Algorithms field shall be set to 1 by a Provisionee. Other sections of this specification may introduce further requirements for the value of this field.

The Public Key Type values are defined in [Table 5.22](#).

Bit	Description
0	Public Key OOB information available
1–7	Reserved for Future Use

Table 5.22: Public Key Type field values

The size of the Public Key OOB information is determined by the selected Algorithm.

The OOB Type values are defined in [Table 5.23](#).

Bit	Description
0	Static OOB information available
1	Only OOB authenticated provisioning supported
2–7	Reserved for Future Use

Table 5.23: OOB Type field values

If bit 1 of the OOB Type field is set to 1, bit 0 of the Algorithms field shall be set to 0 and at least one of the conditions listed below shall be met:

- Bit 0 of the OOB Type field is set to 1.
- The Output OOB Size field is not equal to 0x00.
- The Input OOB Size field is not equal to 0x00.

The maximum size of the Static OOB information is 32 octets. The data type for the Static OOB information shall be Binary, Numeric, or Alphanumeric.

The Output OOB Size defines the number of digits that can be output (e.g., displayed or spoken) when the value Output Numeric is set and Output Alphanumeric is not set in the Output OOB Action field (see [Table 5.25](#)). The Output OOB Size defines the number of digits and uppercase letters that can be output when the value Output Numeric is not set and Output Alphanumeric is set in the Output OOB Action field. The Output OOB Size defines the number of digits and uppercase letters that can be output when the value Output Numeric is set and Output Alphanumeric is set in the Output OOB Action field. The Output OOB Size values are defined in [Table 5.24](#).

Value	Description
0x00	The Provisionee does not support Output OOB
0x01–0x08	Maximum size of Output OOB supported by the Provisionee
0x09–0xFF	Reserved for Future Use

Table 5.24: Output OOB Size field values

The Output OOB Action values are defined in [Table 5.25](#).

Bit	Description	Data Type
0	Blink	Numeric
1	Beep	Numeric
2	Vibrate	Numeric
3	Output Numeric	Numeric
4	Output Alphanumeric	Alphanumeric
5–15	Reserved for Future Use	n/a

Table 5.25: Output OOB Action field values

The Input OOB Size defines the number of digits that can be entered when the value Input Numeric is set and Input Alphanumeric is not set in the Input OOB Action field (see [Table 5.27](#)). The Input OOB Size defines the number of digits and uppercase letters that can be entered when the value Input Numeric is not set and Input Alphanumeric is set in the Input OOB Action field. The Input OOB Size defines the number of digits and uppercase letters that can be entered when the value Input Numeric is set and Input Alphanumeric is set in the Input OOB Action field. The Input OOB Size values are defined in [Table 5.26](#).

Value	Description
0x00	The Provisionee does not support Input OOB
0x01–0x08	Maximum size of Input OOB supported by the Provisionee
0x09–0xFF	Reserved for Future Use

Table 5.26: Input OOB Size field values

The Input OOB Actions are defined in [Table 5.27](#).

Bit	Description	Data Type
0	Push	Numeric
1	Twist	Numeric
2	Input Numeric	Numeric
3	Input Alphanumeric	Alphanumeric
4–15	Reserved for Future Use	n/a

Table 5.27: Input OOB Action field values

5.4.1.3 Provisioning Start

A Provisioner sends a Provisioning Start PDU to indicate the method it has selected from the options in the Provisioning Capabilities PDU. The format of the parameters for the Provisioning Start PDU is defined in Table 5.28.

Field	Size (octets)	Description	Req.
Algorithm	1	The algorithm used for provisioning (see Table 5.29)	M
Public Key	1	Public Key used (see Table 5.30)	M
Authentication Method	1	Authentication Method used (see Table 5.31)	M
Authentication Action	1	Selected Output OOB Action (see Table 5.32) or Input OOB Action (see Table 5.34) or 0x00	M
Authentication Size	1	Size of the Output OOB used (see Table 5.33) or size of the Input OOB used (see Table 5.35) or 0x00	M

Table 5.28: Provisioning Start PDU parameters format

The Algorithm values are defined in Table 5.29.

Value	Description
0x00	BTM_ECDH_P256_CMAC_AES128_AES_CCM
0x01	BTM_ECDH_P256_HMAC_SHA256_AES_CCM
0x02–0xFF	Reserved for Future Use

Table 5.29: Algorithm field values

The Public Key values are defined in Table 5.30.

Value	Description
0x00	No OOB Public Key is used
0x01	OOB Public Key is used
0x02–0xFF	Reserved for Future Use

Table 5.30: Public Key field values

The Authentication Method field values are defined in [Table 5.31](#).

Value	Name	Description
0x00	Authentication with No OOB	No OOB authentication is used
0x01	Authentication with Static OOB	Static OOB authentication is used
0x02	Authentication with Output OOB	Output OOB authentication is used
0x03	Authentication with Input OOB	Input OOB authentication is used
0x04–0xFF	Prohibited	Prohibited

Table 5.31: Authentication Method field values

When the Authentication Method field value is Authentication with No OOB, the Authentication Action field shall be set to 0x00 and the Authentication Size field shall be set to 0x00.

When the Authentication Method field value is Authentication with Static OOB, the Authentication Size shall be set to 0x00 and the Authentication Action field shall be set to 0x00. The data type shall be transferred using an OOB technology.

When the Authentication Method field value is Authentication with Output OOB, the values defined in [Table 5.32](#) and [Table 5.33](#) shall be used to determine the data type, the Authentication Action, and the Authentication Size.

The Output OOB Action values for the Authentication Action field are defined in [Table 5.32](#).

Value	Description	Data Type
0x00	Blink	Numeric
0x01	Beep	Numeric
0x02	Vibrate	Numeric
0x03	Output Numeric	Numeric
0x04	Output Alphanumeric	Alphanumeric
0x05–0xFF	Reserved for Future Use	—

Table 5.32: Output OOB Action values for the Authentication Action field

The Output OOB Size for the Authentication Size field values are defined in [Table 5.33](#).

Value	Description
0x00	Prohibited
0x01–0x08	The Output OOB Size to be used
0x09–0xFF	Reserved for Future Use

Table 5.33: Output OOB Size values for the Authentication Size field

When the Authentication Method field value is Authentication with Input OOB, the values defined in [Table 5.34](#) and [Table 5.35](#) shall be used to determine the data type, the Authentication Action and the Authentication Size.

The Input OOB Action values for the Authentication Action field are defined in [Table 5.34](#).

Value	Description	Data Type
0x00	Push	Numeric
0x01	Twist	Numeric
0x02	Input Numeric	Numeric
0x03	Input Alphanumeric	Alphanumeric
0x04–0xFF	Reserved for Future Use	–

Table 5.34: Input OOB Action values for the Authentication Action field

The Input OOB Size values for the Authentication Size field are defined in [Table 5.35](#).

Value	Description
0x00	Prohibited
0x01–0x08	The Input OOB size to be used
0x09–0xFF	Reserved for Future Use

Table 5.35: Input OOB Size values for the Authentication Size field

5.4.1.4 Provisioning Public Key

The Provisioner sends a Provisioning Public Key PDU to deliver the public key to be used in the ECDH calculations. The format of the parameters for the Provisioning Public Key PDU is defined in [Table 5.36](#).

Field	Size (octets)	Description	Req.
Public Key X	32	The X component of P-256 elliptic curve public key	M
Public Key Y	32	The Y component of P-256 elliptic curve public key	M

Table 5.36: Provisioning Public Key PDU Parameters Format

5.4.1.5 Provisioning Input Complete

The Provisionee sends a Provisioning Input Complete PDU when the user completes the input operation.

There are no parameters for the Provisioning Input Complete PDU.

5.4.1.6 Provisioning Confirmation

The Provisioner or the Provisionee sends a Provisioning Confirmation PDU to its peer to confirm the values exchanged so far, including the OOB Authentication value and the random number that has yet to be exchanged. The format of the parameters for the Provisioning Confirmation PDU is defined in [Table 5.37](#).

Field	Size (octets)	Description	Req.
Confirmation	16 or 32	The values exchanged so far including the OOB Authentication value	M

Table 5.37: Provisioning Confirmation PDU parameters format

If the PDU is sent by the Provisioner, the Confirmation field shall contain the ConfirmationProvisioner value.

If the PDU is sent by the Provisionee, the Confirmation field shall contain the ConfirmationDevice value.

The size of the Confirmation field, the ConfirmationProvisioner value, and the ConfirmationDevice value are defined in Section 5.4.2.4.1.

5.4.1.7 Provisioning Random

The Provisioner or the Provisionee sends a Provisioning Random PDU to enable its peer device to validate the confirmation. The format of the parameters for the Provisioning Random PDU is defined in Table 5.38.

Field	Size (octets)	Description	Req.
Random	16 or 32	The final input to the confirmation	M

Table 5.38: Provisioning Random PDU parameters format

If the PDU is sent by the Provisioner, the Random field shall contain the RandomProvisioner value.

If the PDU is sent by the Provisionee, the Random field shall contain the RandomDevice value.

The size of the Random field, the RandomProvisioner value, and the RandomDevice value are defined in Section 5.4.2.4.1.

5.4.1.8 Provisioning Data

The Provisioner sends a Provisioning Data PDU to deliver provisioning data to a Provisionee. The format of the parameters for the Provisioning Data PDU is defined in Table 5.39.

Field	Size (octets)	Description	Req.
Encrypted Provisioning Data	25	An encrypted and authenticated network key, NetKey Index, Key Refresh Flag, IV Update Flag, current value of the IV Index, and unicast address of the primary element (see Section 5.4.2.5)	M
Provisioning Data MIC	8	PDU Integrity Check value	M

Table 5.39: Provisioning Data PDU parameters format

5.4.1.9 Provisioning Complete

The Provisionee sends a Provisioning Complete PDU to indicate that it has successfully received and processed the provisioning data.

There are no parameters for the Provisioning Complete PDU.

5.4.1.10 Provisioning Failed

The Provisionee sends a Provisioning Failed PDU if it fails to process a received provisioning protocol PDU. The format of the parameters for the Provisioning Failed PDU is defined in [Table 5.40](#).

Field	Size (octets)	Description	Req.
Error Code	1	This represents a specific error in the provisioning protocol encountered by a Provisionee	M

Table 5.40: Provisioning Failed PDU parameters format

The provisioning error codes are defined in [Table 5.41](#).

Value	Name	Description
0x00	Prohibited	Prohibited
0x01	Invalid PDU	The provisioning protocol PDU is not recognized by the Provisionee
0x02	Invalid Format	The arguments of the protocol PDUs are outside expected values or the length of the PDU is different than expected
0x03	Unexpected PDU	The PDU received was not expected at this moment of the procedure
0x04	Confirmation Failed	The computed confirmation value was not successfully verified
0x05	Out of Resources	The provisioning protocol cannot be continued due to insufficient resources in the Provisionee
0x06	Decryption Failed	The Data block was not successfully decrypted
0x07	Unexpected Error	An unexpected error occurred that may not be recoverable
0x08	Cannot Assign Addresses	The Provisionee cannot assign consecutive unicast addresses to all elements
0x09	Invalid Data	The Data block contains values that cannot be accepted because of general constraints
0x0A–0xFF	RFU	Reserved for Future Use

Table 5.41: Provisioning error codes

5.4.1.11 Provisioning Record Request

The Provisioner sends a Provisioning Record Request PDU to request a provisioning record fragment (a part of a provisioning record; see Section 5.4.2.6) from the Provisionee. The format of the parameters for the Provisioning Record Request PDU is defined in Table 5.42.

Field	Size (octets)	Description	Req.
Record ID	2	Identifies the provisioning record for which the request is made (see Section 5.4.2.6).	M
Fragment Offset	2	The starting offset of the requested fragment in the provisioning record data	M
Fragment Maximum Size	2	The maximum size of the provisioning record fragment that the Provisioner can receive	M

Table 5.42: Provisioning Record Request PDU parameters format

A Provisionee may contain multiple provisioning records, and the Record ID field specifies the record that is being requested.

The Fragment Offset field value indicates the starting offset for the provisioning record data fragment in the Provisioning Record Response PDU that corresponds to the Provisioning Record Request. The Fragment Offset field values are in the range 0, indicating the first octet of the provisioning record data, to the provisioning record data length (in bytes) minus 1, indicating the last octet of the provisioning record data.

The Fragment Maximum Size field specifies the maximum fragment size that the Provisioner can accept. The choice of the value is affected by the provisioning bearer that is being used. A value of 0x0000 is Prohibited.

5.4.1.12 Provisioning Record Response

The Provisionee sends a Provisioning Record Response PDU in response to a Provisioning Record Request PDU. The format of the parameters for the Provisioning Record Response PDU is defined in Table 5.43.

Field	Size (octets)	Description	Req.
Status	1	Indicates whether or not the request was handled successfully (see Table 5.44).	M
Record ID	2	Identifies the provisioning record whose data fragment is sent in the response (see Section 5.4.2.6).	M
Fragment Offset	2	The starting offset of the data fragment in the provisioning record data	M
Total Length	2	Total length of the provisioning record data stored on the Provisionee	M

Field	Size (octets)	Description	Req.
Data	N	Provisioning record data fragment	O

Table 5.43: Provisioning Record Response PDU parameters format

Table 5.44 defines status codes for the Provisioning Record Response PDU.

Status Code	Status Code Name
0x00	Success
0x01	Requested Record Is Not Present
0x02	Requested Offset Is Out Of Bounds
0x03–0xFF	Reserved for Future Use

Table 5.44: Status codes for the Provisioning Record Response PDU

The value of the Record ID field of the Provisioning Record Response PDU shall be set to the value of the Record ID field of the corresponding Provisioning Record Request PDU.

The value of the Fragment Offset field of the Provisioning Record Response PDU shall be set to the value of the Fragment Offset field of the corresponding Provisioning Record Request PDU.

If the value of the Status field of the Provisioning Record Response PDU is either Success or Requested Offset Is Out Of Bounds, the Total Length field indicates the length of provisioning record data that the Provisionee has for the provisioning record specified by the Record ID field of the Provisioning Record Response PDU.

If the value of the Status field of the Provisioning Record Response PDU is Requested Record Is Not Present, the Total Length field shall be set to 0x0000.

The Data field contains a fragment of the provisioning record specified by the Record ID field of the Provisioning Record Response PDU.

If the value of the Status field of the Provisioning Record Response PDU is not Success, the Data field shall be empty.

If the value of the Status field of the Provisioning Record Response PDU is Success, the fragment shall start at the position given by the Fragment Offset field of the Provisioning Record Response PDU.

The length of the fragment shall not exceed the value in the Fragment Maximum Size field of the corresponding Provisioning Record Request PDU. The fragment may be shorter than the value of the Fragment Maximum Size field if less data is available or if the Provisionee cannot create a message of the requested size because of memory limitations or some other limiting factor.

5.4.1.13 Provisioning Records Get

The Provisioner sends a Provisioning Records Get PDU to request the list of IDs of the provisioning records that are stored on a Provisionee.

There are no parameters for the Provisioning Records Get PDU.

5.4.1.14 Provisioning Records List

The Provisionee sends a Provisioning Records List PDU in response to a Provisioning Records Get PDU. The format of the parameters for the Provisioning Records List PDU is defined in [Table 5.45](#).

Field	Size (octets)	Description	Req.
Provisioning Extensions	2	Bitmask indicating the provisioning extensions supported by the Provisionee (see Table 5.46)	M
Records List	variable	Lists the Record IDs of the provisioning records stored on the Provisionee (see Section 5.4.2.6)	O

Table 5.45: Provisioning Records List PDU parameters format

[Table 5.46](#) defines provisioning extension values.

Bit	Description
0–15	Reserved for Future Use

Table 5.46: Values for the Provisioning Extensions field

The value of the Provisioning Extensions field of the Provisioning Records List PDU indicates the provisioning extensions supported by the Provisionee.

The Records List field of the Provisioning Records List PDU shall list the 16-bit Record IDs of the provisioning records that are stored on the Provisionee. If no provisioning records are stored on the Provisionee, the field shall be empty.

5.4.2 Provisioning behavior

Provisioning is performed using a five-step process: beaconing, invitation, exchanging public keys, authentication, and distribution of the provisioning data, as illustrated by [Figure 5.15](#).

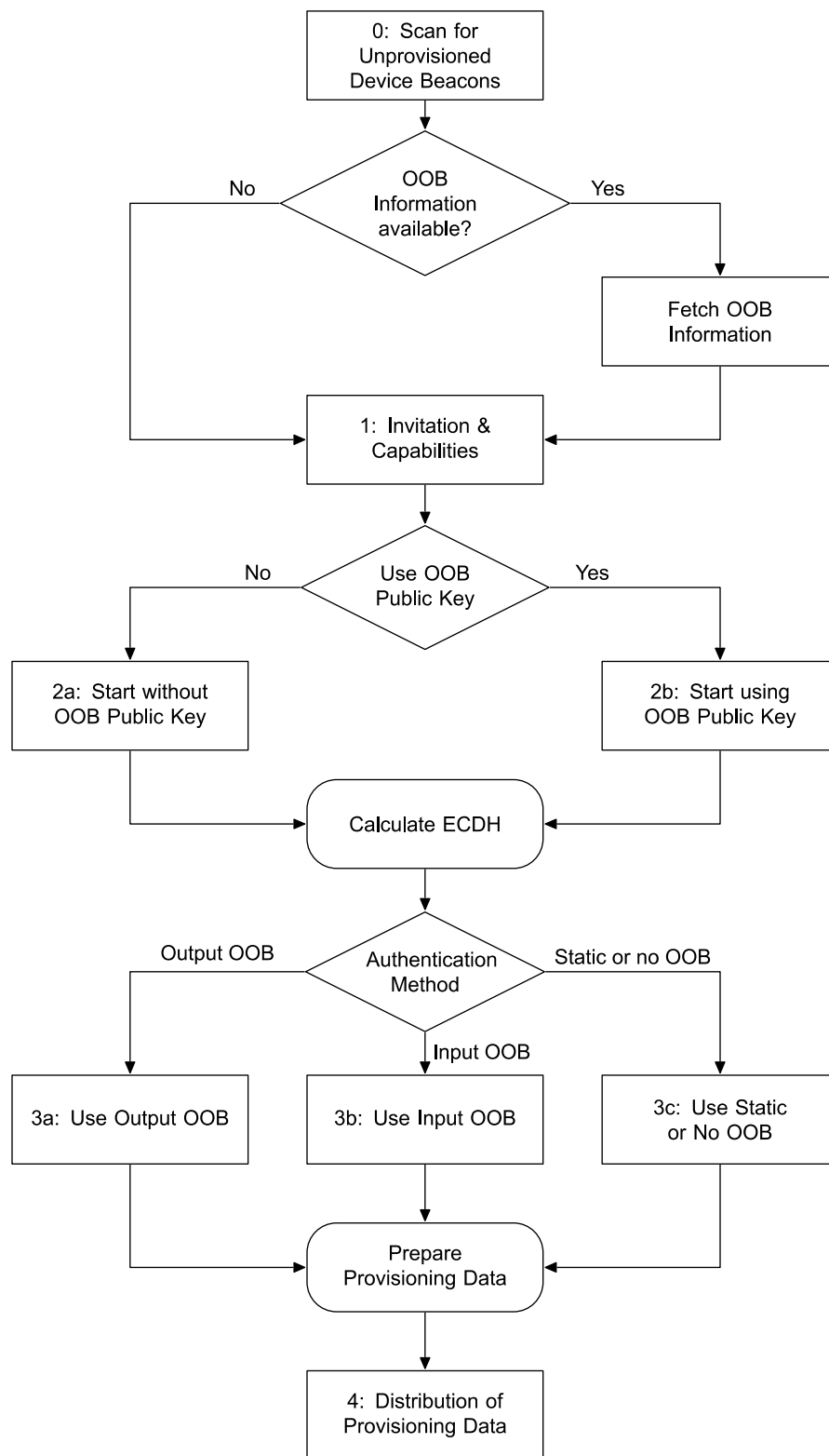


Figure 5.15: Provisioning behavior

5.4.2.1 Beacons

A device that supports PB-ADV, has not been provisioned, and is not in the process of being provisioned, shall advertise the Unprovisioned Device beacon, as defined in Section 3.10.2, otherwise a device shall not advertise the Unprovisioned Device beacon. When a device has not been provisioned, it is recommended to use anonymous advertising [2], a non-resolvable private address, or a resolvable private address. This beacon may indicate availability of OOB data that allows a Provisioner to prompt the user to collect this OOB data before the next step.

A device that supports PB-GATT, has not been provisioned, and is not in the process of being provisioned, must advertise the Mesh Provisioning Service as required by Section 7.1.2.2.1.

5.4.2.2 Invitation

After establishing a provisioning bearer, a Provisioner shall send a Provisioning Invite PDU and the Provisionee shall respond with a Provisioning Capabilities PDU. The Provisioning Invite PDU includes an Attention Duration field for the primary element of the Provisionee and any other elements where the Attention Timer state (described in Section 4.2.10) is present. The Attention Duration field is used to determine how long elements take to identify themselves using the Attention Timer. If the provisioning bearer is dropped, the Provisionee shall set the Attention Timer state of the primary element, and any other elements where the Attention Timer is present, to 0x00 (Off). This Provisioning Capabilities PDU shall include information on the number of supported elements that are present in Composition Data Page 0 (see Section 4.2.2.1), the set of security algorithms supported, the availability of its public key using an OOB technology, the ability for this device to output a value to the user, the ability for this device to allow a value to be input by the user, and if the device has a block of OOB data that can be used for authentication.

Optionally, after establishing a provisioning bearer, a Provisioner may retrieve provisioning record information from the device (see Section 5.4.2.6) before sending a Provisioning Invite PDU.

The message sequence for provisioning invitation is illustrated by Figure 5.16 below.

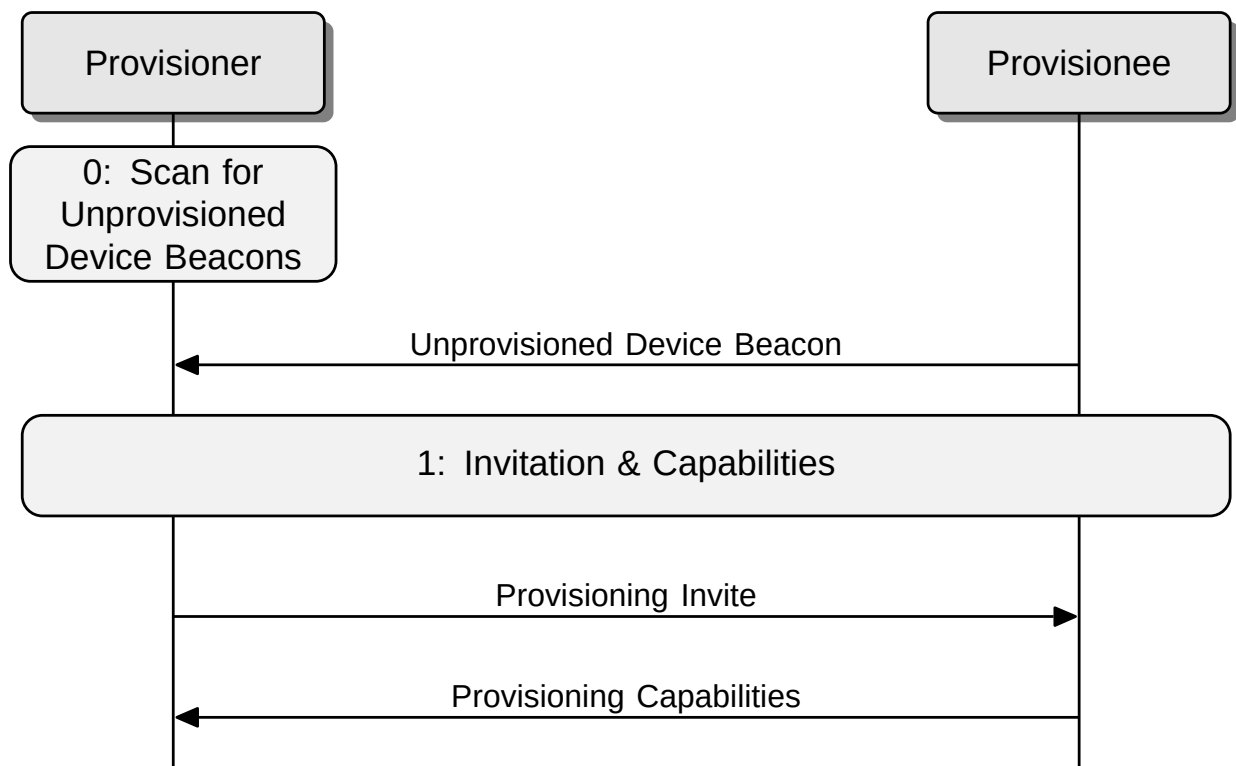


Figure 5.16: Provisioning invitation

5.4.2.3 Exchanging public keys

This step has two possibilities depending on the availability of the Provisionee public key on the Provisioner side. Combined with the three possibilities of the authentication step (see Section 5.4.2.4), there are six possible exchange/authentication paths.

Once the Provisioner has determined that it can provision the Provisionee, it shall send a Provisioning Start PDU that details which of the six possible paths that the Provisioner has chosen to use.

Upon receiving the Provisioning Start PDU from the Provisioner, the Provisionee shall set the Attention Timers present on the device to 0x00.

If bit 1 of the OOB Type field of the Provisioning Capabilities PDU is set to 1 (Only OOB authenticated provisioning supported) and any of the following conditions is met:

- the Algorithm field of the Provisioning Start PDU is not set to BTM_ECDH_P256_CMAC_AES128_AES_CCM and the Provisioning Start PDU has the Authentication Method field set to the Authentication with No OOB value
- the Algorithm field is set to BTM_ECDH_P256_CMAC_AES128_AES_CCM

the provisioning protocol shall fail and the message shall be treated by the Provisionee as an error in the provisioning protocol.

The Provisioner shall support all Algorithm values as defined in [Table 5.29](#). The Provisioner shall select a single algorithm from those offered to it by the New Device in the Provisioning Capability PDU. If the Provisioner does not understand a bit set in this algorithm bit field, it shall ignore that bit and only select from the algorithms it does understand. The Provisioner should choose the strongest algorithm. The Provisioner shall not use BTM_ECDH_P256_CMAC_AES128_AES_CCM if another algorithm is supported by the Provisionee.

If the Provisioner cannot retrieve the public key using an OOB technology or the Provisioner has not accessed the Provisionee's public key via an OOB technology, or if the Provisioner does not use an OOB public key, then the Provisioner sets the Public Key field in the Provisioning Start PDU to zero and the public keys are exchanged between the Provisioner and the Provisionee. For each such exchange, a new key pair shall be generated by the Provisioner and the Provisionee.

The Provisionee shall send its generated public key if the Public Key field in the Provisioning Start PDU is set to zero.

The message sequence for public key exchange when the Provisionee public key is unknown is illustrated by [Figure 5.17](#).

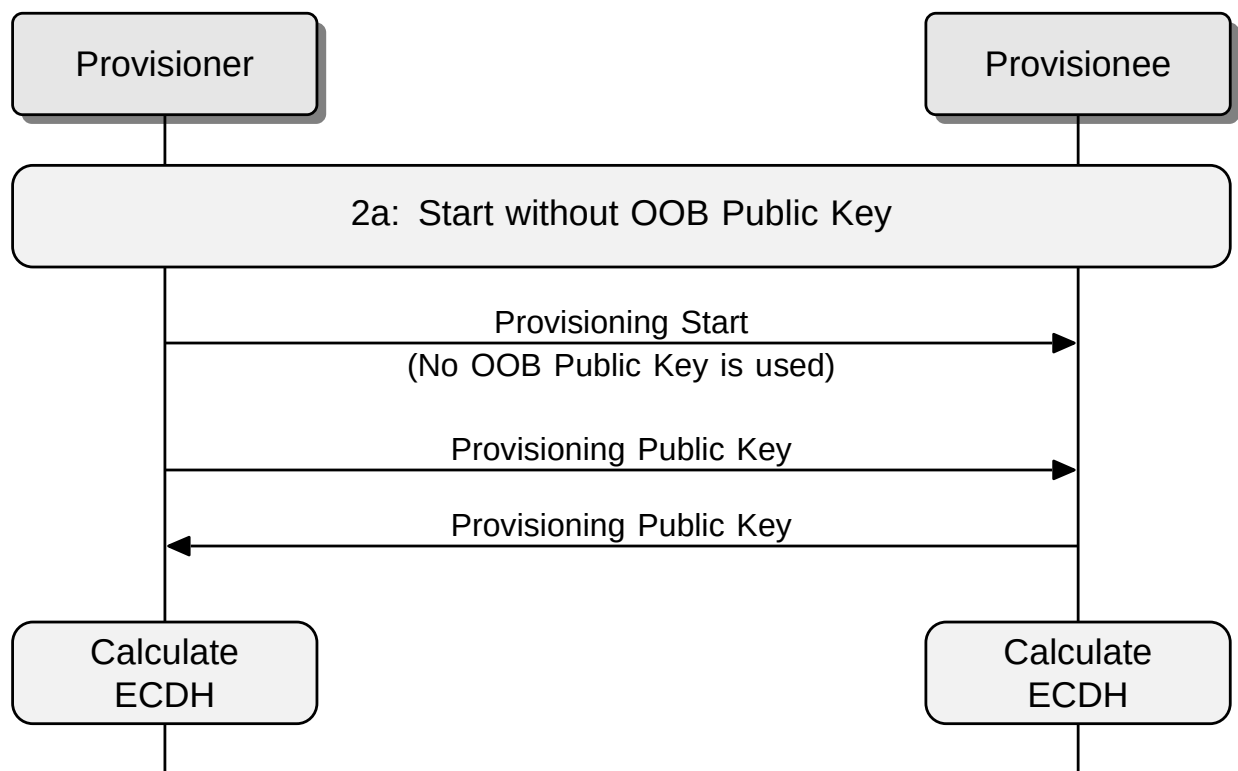


Figure 5.17: Public key exchange when Provisionee public key is unknown

Otherwise, if the public key is available using an OOB technology and the Provisioner has accessed the device's public key via an OOB technology, then the Provisioner sets the Public Key field in the Provisioning Start PDU to 1, and a new key pair shall be generated by the Provisioner, and the public key of the generated key pair shall be transmitted from the Provisioner to the Provisionee, and the public key read from the device using the appropriate OOB technology shall be used.

A Provisionee's public key that is contained in a Device Certificate as specified in Section 5.5.1, and retrievable from the device over an established provisioning bearer (see Section 5.4.2.6) or retrievable from the Internet (see Section 5.6), is considered to be available using an OOB technology when the Provisioner supports certificate-based provisioning.

The message sequence for public key exchange when the unprovisioned device public key is OOB is illustrated by Figure 5.18.

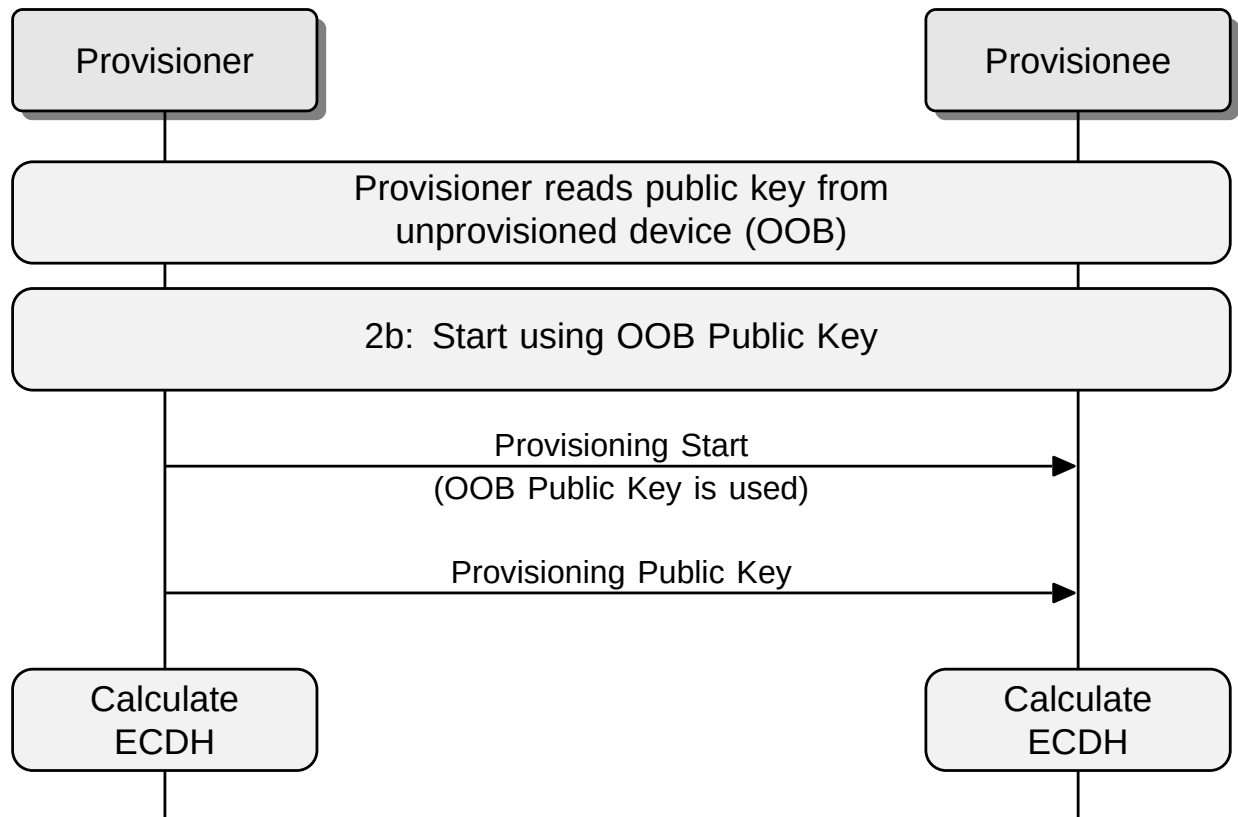


Figure 5.18: Public key exchange when Provisionee public key is OOB

The Provisioner and the Provisionee shall check whether the public key provided by the peer device or obtained OOB is valid (see Section 5.4.3.1).

When the Provisioner receives an invalid public key or a public key that is equal to the Provisioner public key, then provisioning shall fail, and the Provisioner shall act as described in Section 5.4.4. When the Provisionee receives an invalid public key or a public key that is equal to the Provisionee public key, then provisioning shall fail and the Provisionee shall act as described in Section 5.4.4 and should indicate the Invalid Format Error Code in the Provisioning Failed PDU.

After the public key is known and has been validated, the ECDHSecret shall be computed using the following formula:

$$ECDHSecret = P256(\text{private key}, \text{peer public key})$$

After the ECDHSecret is computed, the Provisioner and the Provisionee shall delete its private-public key pair that was generated in this step.

5.4.2.4 Authentication

5.4.2.4.1 AuthValue computation

Once the public key exchange has been performed, a confirmation exchange followed by a random number exchange is performed. The confirmation values are a cryptographic hash of all the values exchanged so far, the random number that is yet to be revealed, and the AuthValue. Once the random numbers are exchanged, each device authenticates its peer.

The authentication value from the Authentication Method (Authentication with Output OOB, Authentication with Input OOB, or Authentication with Static OOB) is used in the AuthValue for the purpose of computing the confirmation value. If the Algorithm field is BTM_ECDH_P256_CMAC_AES128_AES_CCM, the confirmation value of the Provisioner is a 128-bit value, the confirmation value of the Provisionee is a 128-bit value, and they are computed using:

$$ConfirmationProvisioner = AES-CMAC_{ConfirmationKey} (RandomProvisioner || AuthValue)$$

$$ConfirmationDevice = AES-CMAC_{ConfirmationKey} (RandomDevice || AuthValue)$$

Where:

$$ConfirmationKey = k1(ECDHSecret, ConfirmationSalt, "prck")$$

$$ConfirmationSalt = s1(ConfirmationInputs)$$

$$ConfirmationInputs = ProvisioningInvitePDUValue || ProvisioningCapabilitiesPDUValue || ProvisioningStartPDUValue || PublicKeyProvisioner || PublicKeyDevice$$

If the Algorithm field is BTM_ECDH_P256_HMAC_SHA256_AES_CCM, the confirmation value of the Provisioner is a 256-bit value, the confirmation value of the Provisionee is a 256-bit value, and they are computed using:

$$ConfirmationProvisioner = HMAC-SHA-256_{ConfirmationKey} (RandomProvisioner)$$

$$ConfirmationDevice = HMAC-SHA-256_{ConfirmationKey} (RandomDevice)$$

Where:

$$ConfirmationKey = k5(ECDHSecret || AuthValue, ConfirmationSalt, "prck256")$$

$$ConfirmationSalt = s2(ConfirmationInputs)$$

$$ConfirmationInputs = ProvisioningInvitePDUValue || ProvisioningCapabilitiesPDUValue || ProvisioningStartPDUValue || PublicKeyProvisioner || PublicKeyDevice$$

The ProvisioningInvitePDUValue is the value of the Provisioning Invite PDU fields (excluding the opcode) that was sent or received.



The ProvisioningCapabilitiesPDUValue is the value of the Provisioning Capabilities PDU fields (excluding the opcode) that was sent or received.

The ProvisioningStartPDUValue is the value of the Provisioning Start PDU fields (excluding the opcode) that was sent or received.

The PublicKeyProvisioner is the value of the Public Key X and Public Key Y fields from the Public Key PDU that was sent by the Provisioner.

The PublicKeyDevice is the value of the Public Key X and Public Key Y fields from the Public Key PDU that was sent by the Provisionee or from the delivered OOB public key.

The RandomProvisioner is a string of random bits generated by the Provisioner's random number generator. The random number generator shall be compatible with the requirements in Volume 2, Part H, Section 2 of the Core Specification [1]. If the Algorithm field is BTM_ECDH_P256_CMAC_AES128_AES_CCM, the RandomProvisioner is a 128-bit value. If the Algorithm field is BTM_ECDH_P256_HMAC_SHA256_AES_CCM, the RandomProvisioner is a 256-bit value.

The RandomDevice is a string of random bits generated by the Provisionee's random number generator. The random number generator shall be compatible with the requirements in Volume 2, Part H, Section 2 of the Core Specification [1]. If the Algorithm field is BTM_ECDH_P256_CMAC_AES128_AES_CCM, the RandomDevice is a 128-bit value. If the Algorithm field is BTM_ECDH_P256_HMAC_SHA256_AES_CCM, the RandomDevice is a 256-bit value.

If the Algorithm field is BTM_ECDH_P256_CMAC_AES128_AES_CCM, the AuthValue is a 128-bit value. If the Algorithm field is BTM_ECDH_P256_HMAC_SHA256_AES_CCM, the AuthValue is a 256-bit value. The computation of AuthValue depends on the data type. There are three data type values defined: Binary, Numeric, and Alphanumeric.

If the Authentication Method field value is Authentication with No OOB, the data type is Numeric, and the authentication value is 0.

If the Authentication Method field value is Authentication with Static OOB, the data type and authentication value are transferred using an OOB technology.

If the Authentication Method field value is Authentication with Output OOB, the data type is defined by the Output OOB Action (see Table 5.32) and the authentication value is transferred using the mechanism defined by the Authentication Action.

If the Authentication Method field value is Authentication with Input OOB, the data type is defined by the Input OOB Action (see Table 5.34) and the authentication value is transferred using the mechanism defined by the Authentication Action.

Data type Binary If the data type is Binary, the authentication value is an array of octets. The authentication value array shall be copied to the AuthValue. If the authentication value is shorter than the defined AuthValue size, the remaining bits shall be set to 0. If the authentication value is longer than the defined AuthValue size, the array shall be trimmed by removing octets with indexes higher than needed.

For example, if the Algorithm field is BTM_ECDH_P256_CMAC_AES128_AES_CCM and the Authentication with Output OOB method is used with the Output OOB Action as Output Alphanumeric and the displayed string is "123ABC" (authentication value), then the AuthValue shall be

0x31323341424300000000000000000000, resulting in an array consisting of [0x31, 0x32, 0x33, 0x41, 0x42, 0x43, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00].

5.4.2.4.2 Confirmation and random number exchange

To complete confirmation and random number exchange, the Provisioner and the Provisionee shall follow these steps:

1. When the confirmation value of the Provisioner is computed, the Provisioner shall send the ConfirmationProvisioner value to the Provisionee using the Provisioning Confirmation PDU.
2. When the Provisionee receives the Provisioning Confirmation PDU from the Provisioner, it shall compare the ConfirmationProvisioner and the ConfirmationDevice values.
 - If the values are not equal, the Provisionee shall send the ConfirmationDevice value to the Provisioner using the Provisioning Confirmation PDU.
 - If the values are equal, the provisioning protocol shall fail, and the Provisionee shall act as described in Section 5.4.4 and should indicate the Invalid Format Error Code in the Provisioning Failed PDU.
3. When the Provisioner receives the Provisioning Confirmation PDU from the Provisionee, it shall compare the ConfirmationProvisioner and the ConfirmationDevice values.
 - If the values are not equal, the Provisioner shall send the RandomProvisioner value to the Provisionee using the Provisioning Random PDU.
 - If the values are equal, the provisioning protocol shall fail, and the Provisioner shall act as described in Section 5.4.4.
4. When the Provisionee receives the Provisioning Random PDU from the Provisioner, it shall compute the ConfirmationProvisioner value by using the received RandomProvisioner value.
 - If the ConfirmationProvisioner value received is equal to the computed ConfirmationProvisioner, the Provisioner is authenticated, and the Provisionee shall send the RandomDevice value to the Provisioner using the Provisioning Random PDU.
 - If the ConfirmationProvisioner value received is not equal to the computed ConfirmationProvisioner, the provisioning protocol shall fail, and the Provisionee shall act as described in Section 5.4.4 and should indicate the Confirmation Failed Error Code in the Provisioning Failed PDU.
5. When the Provisioner receives the Provisioning Random PDU from the Provisionee, it shall compute the ConfirmationDevice value by using the received RandomDevice value.
 - If the ConfirmationDevice value received is equal to the computed ConfirmationDevice, the Provisionee is authenticated.
 - If the ConfirmationDevice value received is not equal to the computed ConfirmationDevice, the provisioning protocol shall fail, and the Provisioner shall act as described in Section 5.4.4.



5.4.2.4.3 Authentication with Output OOB

If Output OOB authentication is used, then the Provisioning Start PDU would include a request to output either a single-digit or multiple-digit value on the Provisionee device by using a prescribed action. Examples of output actions may include making a noise, blinking a light, voice, or displaying symbols on a display.

For example, if the unprovisioned device is a door lock that includes an LED, then it would be possible to use the Output OOB authentication with the action that would blink that LED.

When the Authentication Method field value is Authentication with Output OOB and when the Output OOB Action for the Authentication Action value is equal to Blink, Beep, or Vibrate, then the Provisionee shall select a random integer between 1 and $10^{\text{Authentication Size}} - 1$ inclusive. That random number shall be output as a sequence of events (e.g., by blinking, beeping, or vibrating with a duty cycle of 500 milliseconds on and 500 milliseconds off) with a gap of at least 3 seconds between sequences to allow the user to determine the end of the sequence. When the Output OOB Action for the Authentication Action value is equal to Output Numeric, then the value is output (e.g., displayed or spoken) using a number of digits (i.e., ASCII character codes 0x30 to 0x39) determined by the Authentication Size value. When the Output OOB Action for the Authentication Action value is equal to Output Alphanumeric, then the value is output using a number of ASCII digits and uppercase letters (i.e., ASCII character codes 0x30 to 0x39 and 0x41 to 0x5A) determined by the Authentication Size value.

The user of the Provisioner shall input the number observed to authenticate that Provisionee.

Once input of the number has been performed, a confirmation exchange followed by a random number exchange is performed (see Section 5.4.2.4.2). The confirmation values are a cryptographic hash of all the values exchanged so far, the random number that is yet to be revealed, and the number that has been input. Once the random numbers are exchanged, each device can authenticate its peer.

The message sequence for Authentication with Output OOB is illustrated by [Figure 5.19](#).

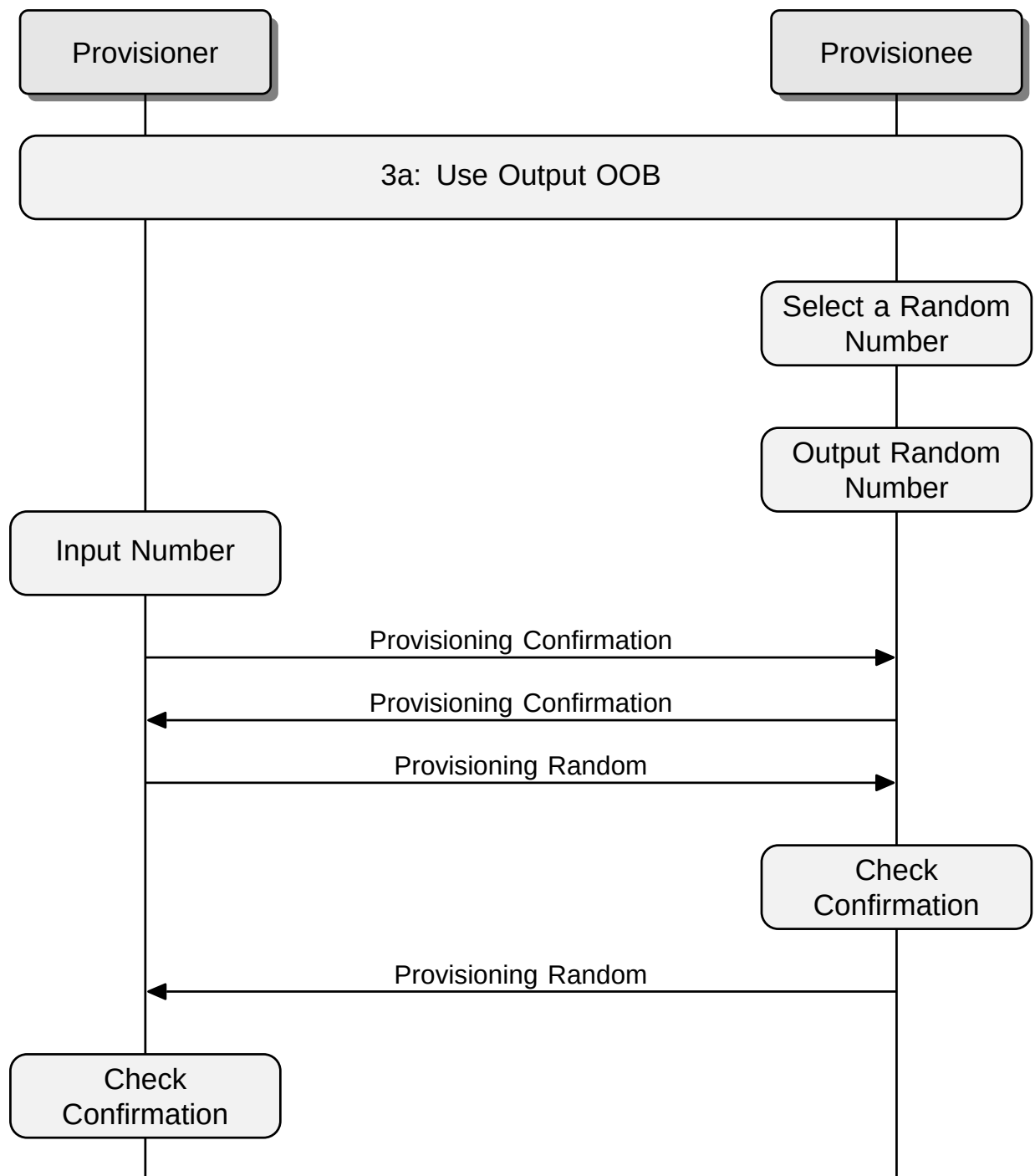


Figure 5.19: Authentication with Output OOB

5.4.2.4.4 Authentication with Input OOB

When the Authentication Method field value is Authentication with Input OOB and when the Input OOB Action for the Authentication Action value is equal to Push, then the Provisioner shall select a random integer between 1 and $10^{\text{Authentication Size}} - 1$ inclusive. That random number shall be input by the number of

push actions. When the Input OOB Action for the Authentication Action value is equal to Twist, then the Provisioner shall select a random integer between 1 and $10^{\text{Authentication Size}} - 1$ inclusive. That random number shall be input by the number of twist actions until the value on the control has been entered. When the Input OOB Action for the Authentication Action value is equal to Input Numeric, the value is input by entering a number of digits (probably using a numeric keyboard) determined by the Authentication Size value. When the Input OOB Action for the Authentication Action value is equal to Input Alphanumeric, the value is input by entering a number of ASCII digits and uppercase letters (probably using an alphanumeric keyboard) determined by the Authentication Size value.

The Provisioner shall then prompt the user to input that value into the Provisionee device by using an appropriate action.

When the Input OOB Action for the Authentication Action value is equal to Push or Twist, the value is considered to have been input after no further input actions are detected for more than 5 seconds. When the Input OOB Action for the Authentication Action value is equal to Input Numeric or Input Alphanumeric, the value is considered to have been input locally on that device (e.g., by pressing an Enter key).

For example, if the unprovisioned device is a light switch, then it would allow the user to input the random number by pressing a button an appropriate number of times.

Once input of the number has been performed, the Provisionee shall send the Provisioning Input Complete PDU to the Provisioner to confirm that the Provisionee has an input value. A confirmation exchange followed by a random number exchange is performed (see Section 5.4.2.4.2). The confirmation values are a cryptographic hash of all the values exchanged so far, the random number that is yet to be revealed, and the number that has been input. Once the random numbers are exchanged, each device can authenticate its peer.

The message sequence for Authentication with Input OOB is illustrated by [Figure 5.20](#).

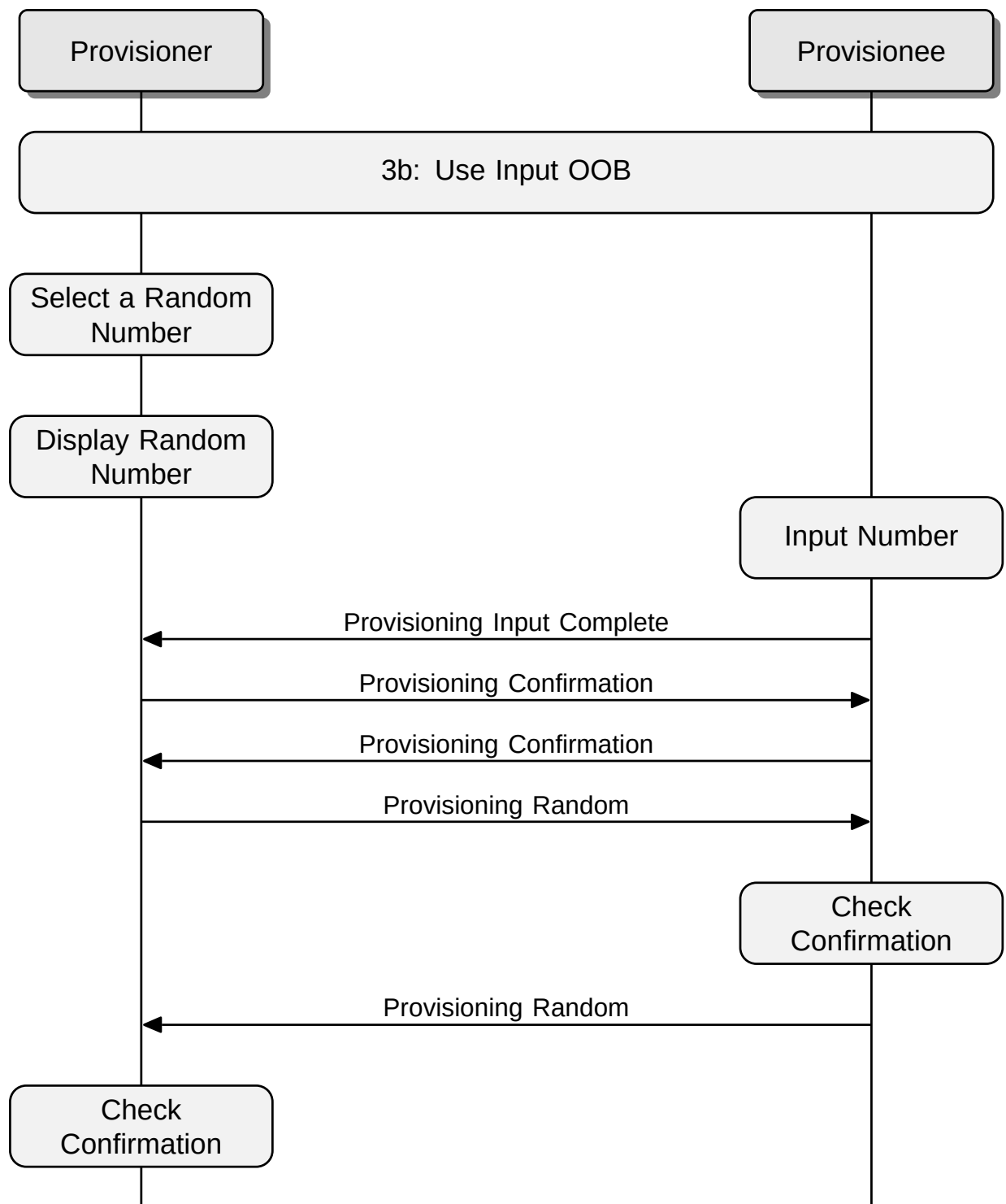


Figure 5.20: Authentication with Input OOB

5.4.2.4.5 Authentication with No OOB and Authentication with Static OOB

When the Authentication Method field value is Authentication with No OOB or the Authentication Method field value is Authentication with Static OOB then the Provisioner shall immediately use the confirmation and random number exchanges (see Section 5.4.2.4.2). If a static OOB value is available, then this value shall be included as part of the confirmation value. If no static OOB value is available, then this value shall have a value of zero.

The message sequence for Authentication with Static OOB or Authentication with No OOB is illustrated by Figure 5.21.

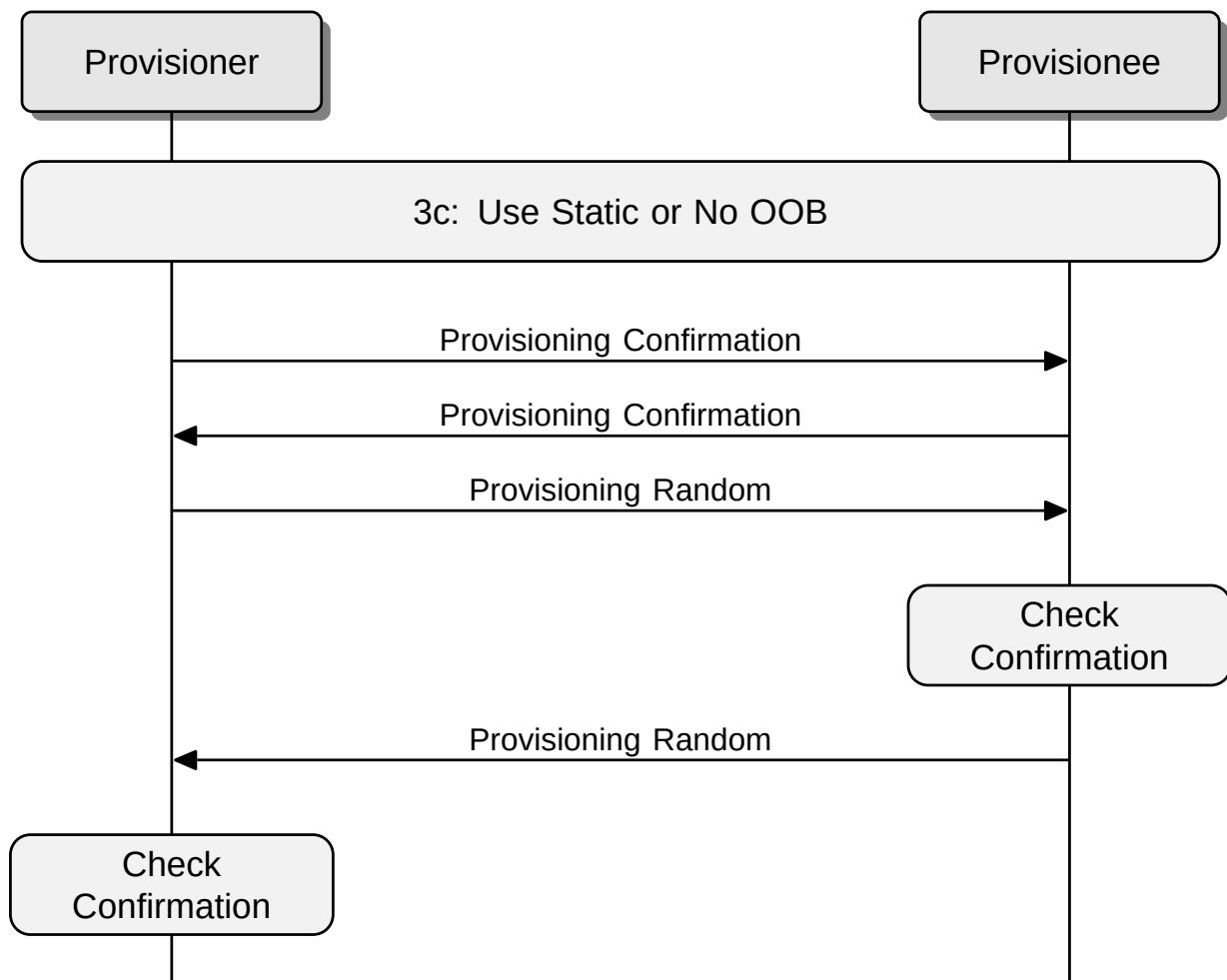


Figure 5.21: Authentication with Static OOB or Authentication with No OOB

5.4.2.5 Distribution of provisioning data

Once the Provisionee has been authenticated, the Provisioner and Provisionee shall use the calculated Diffie-Hellman shared secret ECDHSecret and generate a session key from that shared secret. That session key shall then be used to encrypt and authenticate the provisioning data. The Provisioner then shall send the Provisioning Data PDU containing the encrypted and authenticated provisioning data to the Provisionee.

The provisioning data format is described in [Table 5.47](#).

Field	Size (octets)	Description	Req.
Network Key	16	NetKey	M
Key Index	2	Index of the NetKey	M
Flags	1	Flags bitmask	M
IV Index	4	Current value of the IV Index	M
Unicast Address	2	Unicast address of the primary element	M

Table 5.47: Provisioning data format

The Network Key shall contain the NetKey.

The Key Index field shall identify the global NetKey Index of the NetKey and shall be encoded as defined in Section [4.3.1.1](#).

The values of the Flags field are defined in [Table 5.48](#).

Bits	Definition
0	Key Refresh Flag 0: Key Refresh Phase 0 1: Key Refresh Phase 2
1	IV Update Flag 0: Normal Operation 1: IV Update in Progress
2–7	Reserved for Future Use

Table 5.48: Flags field definition

The IV Index field shall contain the current value of the IV Index.

The Unicast Address shall contain the Unicast Address of the primary element of the node being added to the network.

The Session key shall be derived using the formula:

$$ProvisioningSalt = s1(ConfirmationSalt || RandomProvisioner || RandomDevice)$$

$$SessionKey = k1(ECDHSecret, ProvisioningSalt, "prsk")$$

The nonce shall be the 13 least significant octets of:

$$SessionNonce = k1(ECDHSecret, ProvisioningSalt, "prsn")$$

The provisioning data shall be encrypted and authenticated using:

Provisioning Data = *Network Key* // *Key Index* // *Flags* // *IV Index* // *Unicast Address*

Encrypted Provisioning Data, Provisioning Data MIC = $\text{AES-CCM}_{\text{SessionKey}}(\text{SessionNonce}, \text{Provisioning Data})$

The size of the Provisioning Data MIC is 8 octets.

The Encrypted Provisioning Data and Provisioning Data MIC shall be used as fields in the Provisioning Data PDU. The Provisioner shall send the Provisioning Data PDU to the Provisionee.

The Provisionee shall then compute the device key as defined in Section 3.9.6.1.

The message sequence for distribution of provisioning data is illustrated by Figure 5.22 below.

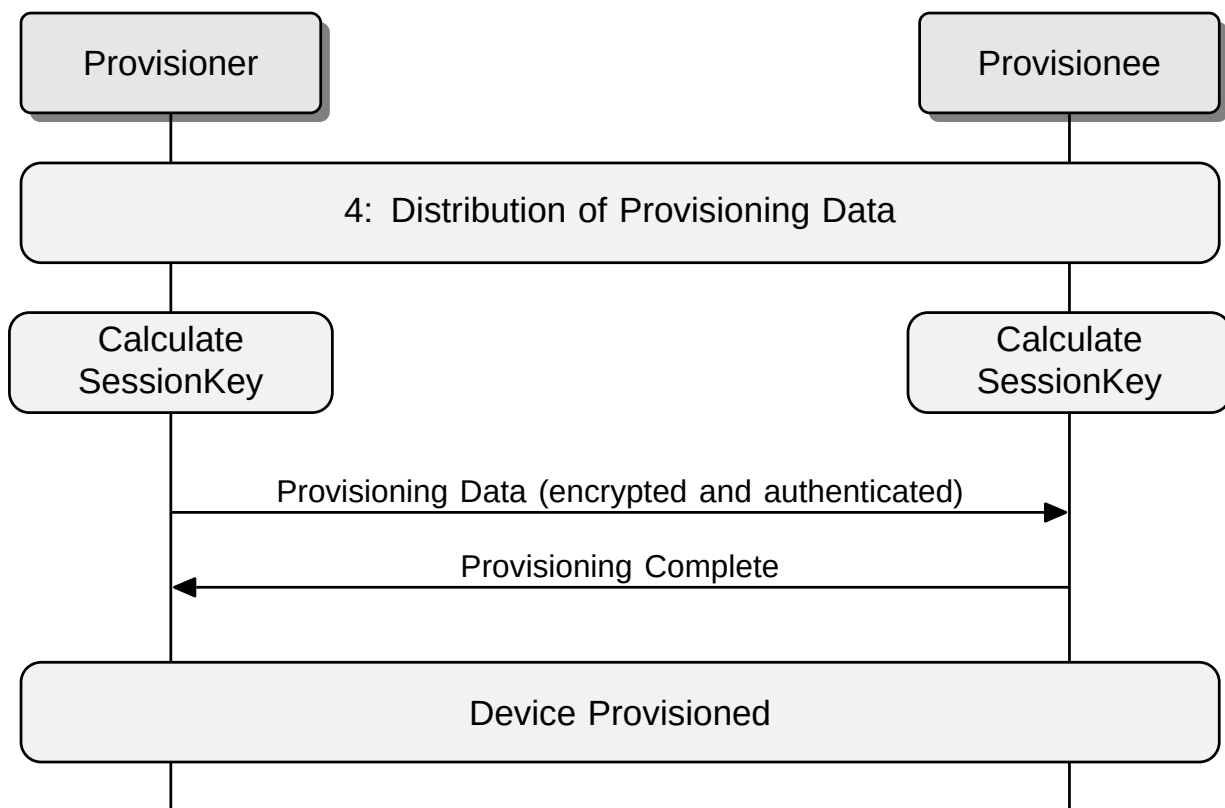


Figure 5.22: Distribution of provisioning data

The Provisioner and Provisionee calculate the device key using the k1 key derivation function based on the established Diffie-Hellman shared secret ECDHSecret.

Upon receiving the Provisioning Data PDU from the Provisioner, the Provisionee shall decrypt and authenticate the provisioning data. Upon successful authentication of the provisioning data, the Provisionee shall set the network key (identified by the Key Index), set the IV Index, set the IV Update procedure state based on the IV Update Flag, set the Key Refresh Phase based on the Key Refresh Flag, and assign unicast addresses to all device elements starting from the primary element and using a

consecutive range of addresses starting from the provided unicast address value. Upon successful completion of the address assigning procedure, the Provisionee shall respond with a Provisioning Complete PDU to confirm that it has been provisioned. If the address assigning cannot be successfully completed, the Provisionee shall assume that provisioning failed and respond with a Provisioning Failed PDU with the Error Code parameter set to Cannot Assign Addresses.

Note: After processing the Provisioning Data PDU from the Provisioner, the 96-hour time limits for changing the IV Update procedure state, as defined in the IV Update procedure, do not apply.

Upon receiving the Provisioning Complete PDU from the Provisionee, the Provisioner shall assume that the provisioning process is completed successfully and the Provisionee is using a consecutive range of address starting from the value of the unicast address. The length of the address range is reported to the Provisioner in Provisioning Capabilities PDU (see Section 5.4.1.2). As a final step in procedure, the Provisioner shall disconnect the provisioning bearer. The Provisionee is now a node in the mesh network.

The Provisioner shall not reuse unicast addresses that have been allocated to a Provisionee and sent in a Provisioning Data PDU unless the node to which the unicast addresses were previously assigned has been removed from the network and the current IV Index (in use during the Node Removal procedure) has been updated, as required in Section 3.11.7, or the Node Address Refresh procedure was executed and the current IV Index (in use during the Node Address Refresh procedure) has been updated, as required in Section 3.11.8.5.

5.4.2.6 Provisioning record retrieval over a provisioning bearer

Provisioning records are read-only data items that the Provisioner may retrieve from the Provisionee after establishing a provisioning bearer. The Provisioner may use the information contained within provisioning record data to manage the provisioning process. Provisioning records are defined in Section 5.4.2.6.3.

Each provisioning record is identified by a 16-bit Record ID, which is an assigned number. See Table 5.52 for the defined Record IDs.

5.4.2.6.1 Provisioning record list retrieval

The Provisioner may retrieve the list of the Record IDs of the provisioning records stored on the Provisionee over an established provisioning bearer by sending a Provisioning Records Get PDU (see Section 5.4.1.13).

To retrieve the Record ID list, the Provisioner shall send a Provisioning Records Get PDU before it sends a Provisioning Invite PDU. A Provisionee that supports provisioning records shall respond to a Provisioning Records Get PDU with a Provisioning Records List PDU. The Provisioner shall not send a new PDU until it has received a Provisioning Records List PDU in response to the previously sent request.

When a Provisionee receives a Provisioning Records Get PDU, and the Provisioning Invite PDU has not already been received after the most recent establishment of a provisioning bearer, then the Provisionee shall respond with a Provisioning Records List PDU with the fields set as defined in Section 5.4.1.14.

When a Provisionee receives a Provisioning Records Get PDU, and the Provisioning Invite PDU has already been received after the most recent establishment of a provisioning bearer, this indicates that